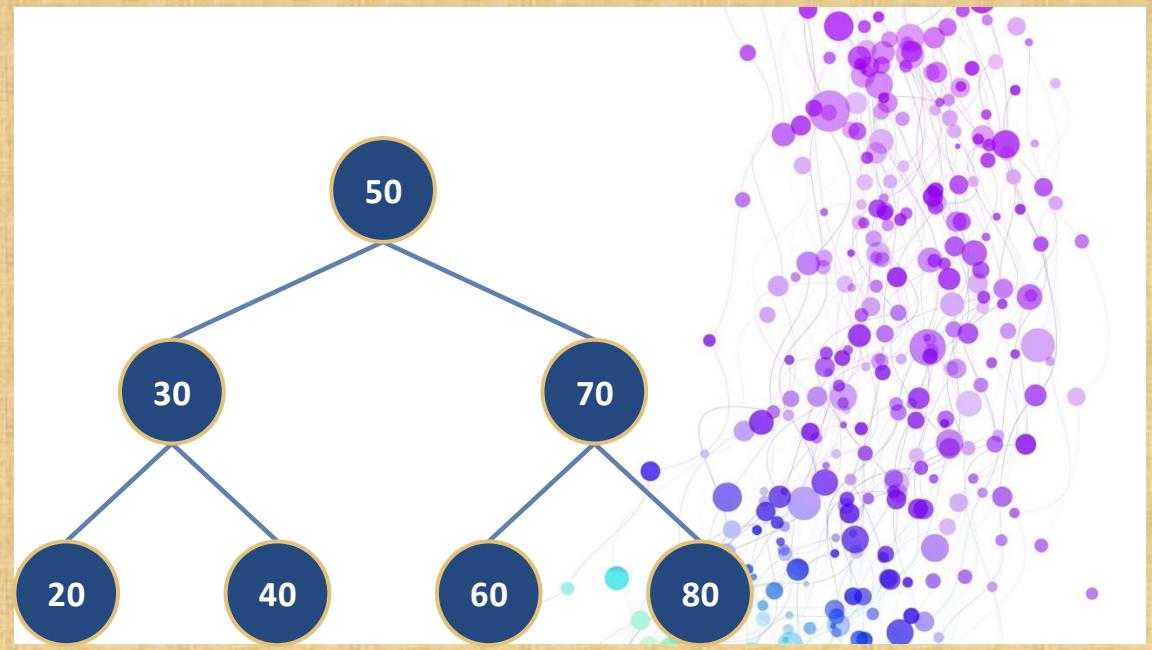
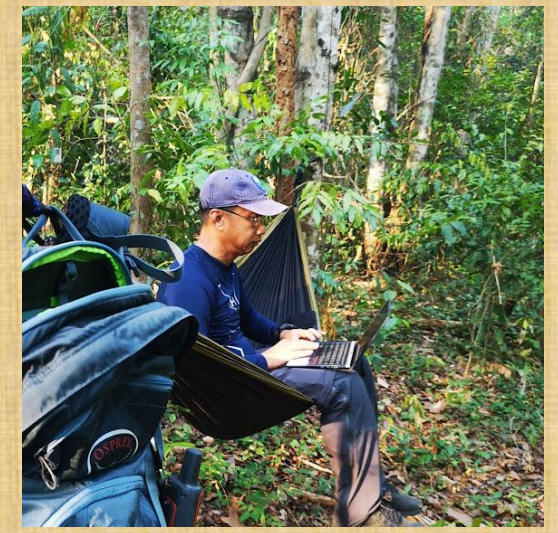
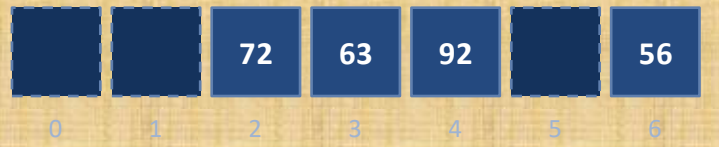
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS

Lecture 9 Trees, Binary Search Trees, and Hash Tables







Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
2026/1

Mahidol University
Faculty of Information and Communication Technology

LO1–LO8 New

By the end of today you will be able to…

LO1 Identify the parts and structural properties of trees and binary trees

LO2 Trace preorder, inorder and postorder traversals

LO3 Search, insert and delete in a binary search tree

LO4 Explain why BST cost depends on the tree's height

LO5 Describe hashing and explain why collisions occur

LO6 Demonstrate chaining, linear & quadratic probing, double hashing

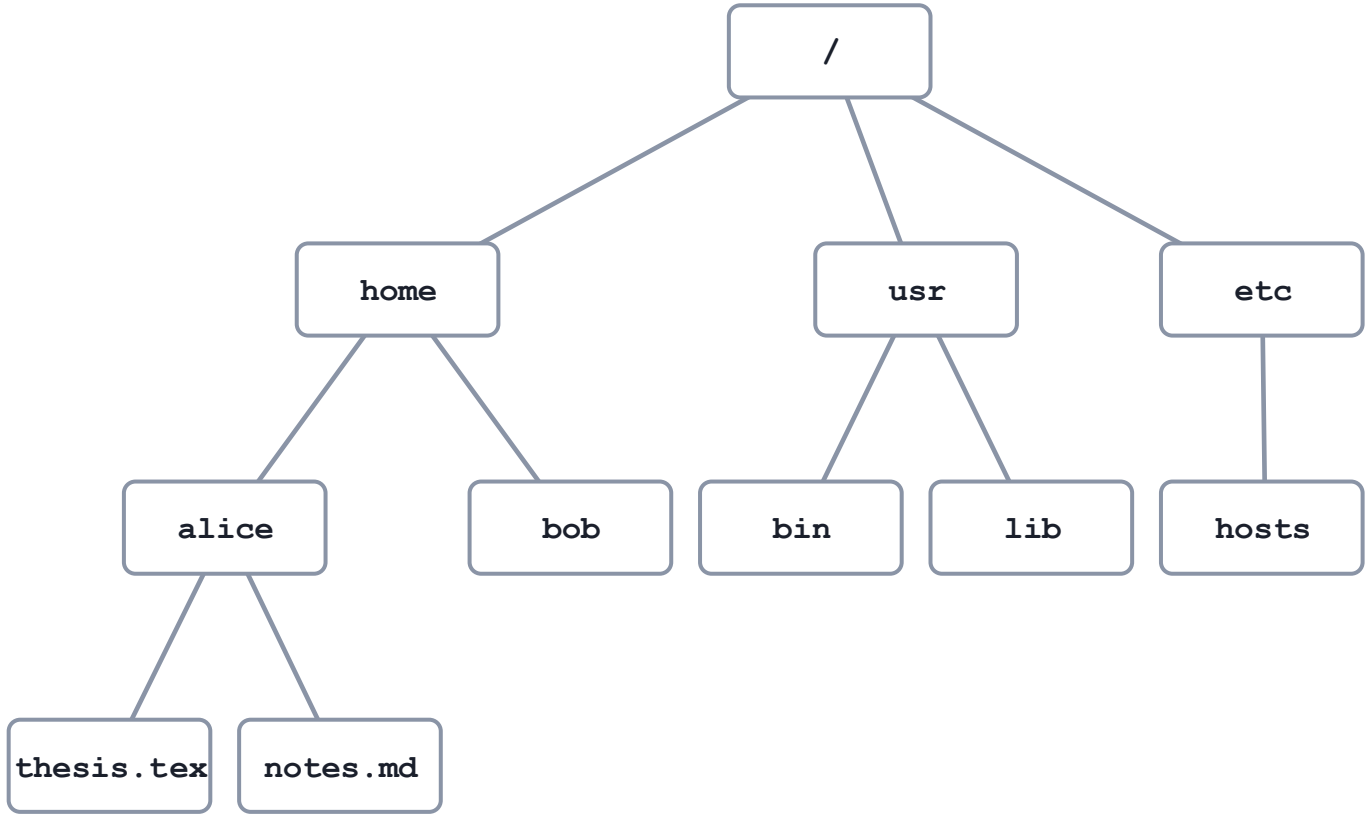
LO7 Find and fix common bugs in Java hash tables

LO8 Compare BSTs and hash tables and choose between them



Tree Fundamentals

Why trees? Hierarchies are everywhere



A file system is a tree: every folder except / has exactly one parent.

Linear structures

Array, linked list: each element has one 'next'. Searching may visit all n elements.

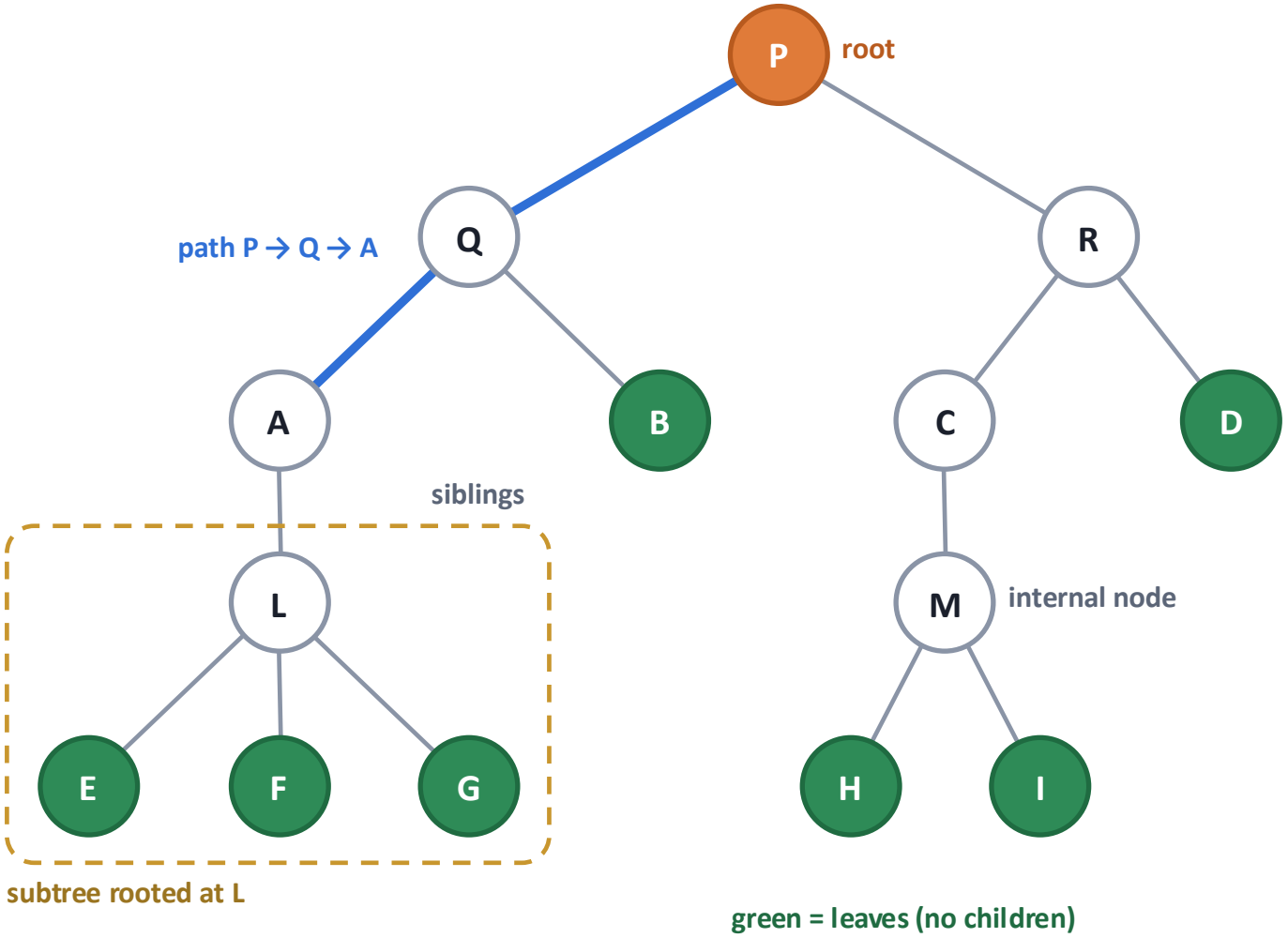
Trees (non-linear)

One parent, any number of children. Natural for hierarchy: folders, org charts, HTML, expressions.

The payoff (Section C)

A well-shaped tree with n keys reaches any key in about $\log_2 n$ steps: 1,000,000 keys → ~20 steps.

Anatomy of a tree



Node: holds a key (and possibly data)

Edge: connects a parent to a child

Root: the only node with no parent

Parent / child: u is the parent of v if an edge goes down from u to v

Siblings: nodes with the same parent

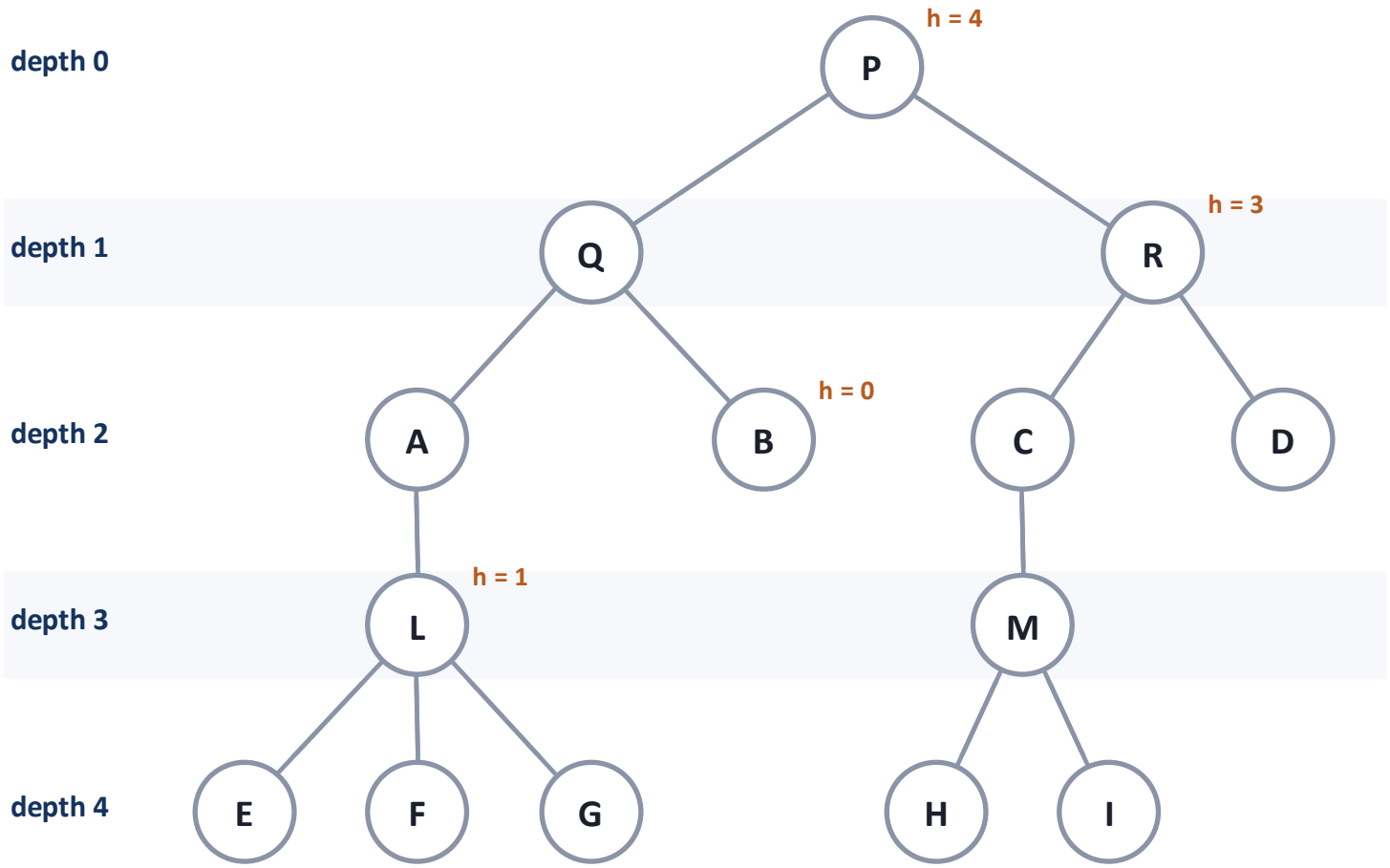
Leaf: node with no children

Internal node: node with ≥ 1 child

Path: sequence of nodes joined by edges; length = #edges

Subtree: a node together with all its descendants

Measuring a tree: depth, height, degree



Definitions (edges, not nodes)

depth(v) = # edges from root to v

height(v) = # edges on the longest path from v down to a leaf

height(tree) = height(root) = maximum depth

degree(v) = number of children of v

Convention used in this course

height(leaf) = 0

height(empty tree) = −1

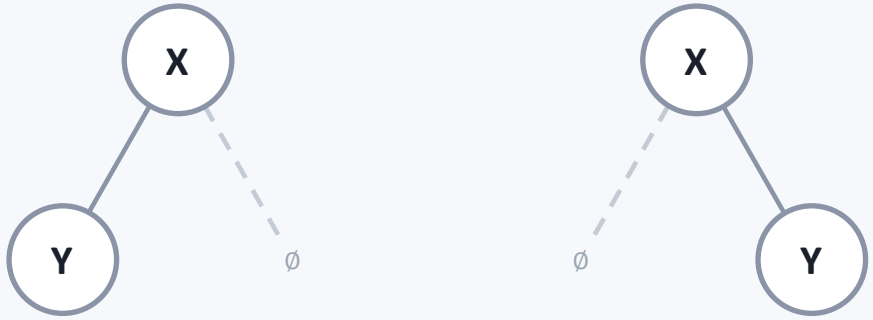
so height(v) = 1 + max(height(left), height(right))

Here: height(tree) = 4 (e.g. P→R→C→M→H), degree(L) = 3, depth(M) = 3

Binary trees: at most two children, left ≠ right

A binary tree is either EMPTY, or a root + a LEFT binary tree + a RIGHT binary tree

Left and right are different positions

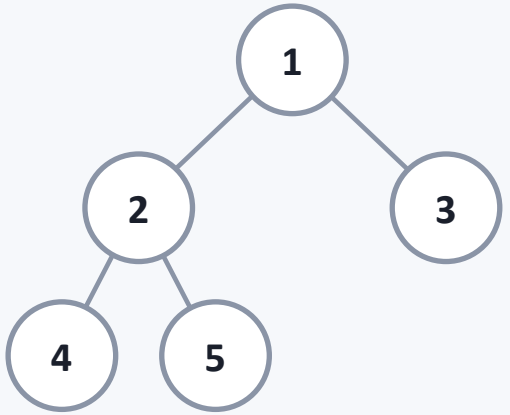


Y is a left child

Y is a right child

Two different binary trees — but the same general tree.

A binary tree need not be ordered



Valid binary tree ✓

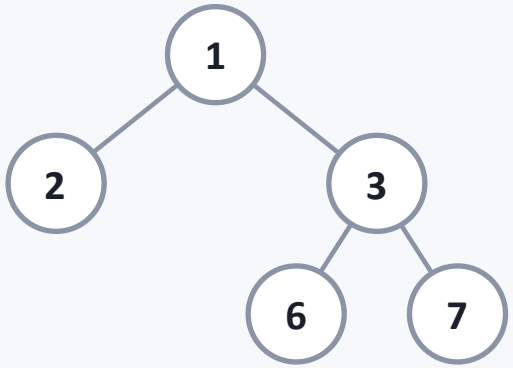
But not a BST: 2 sits left of 1 although 2 > 1.

Inorder gives 4 2 5 1 3 — not sorted.

Ordering (BST property) is an extra rule we add in Section C.

Full, complete, perfect: three different properties

Full

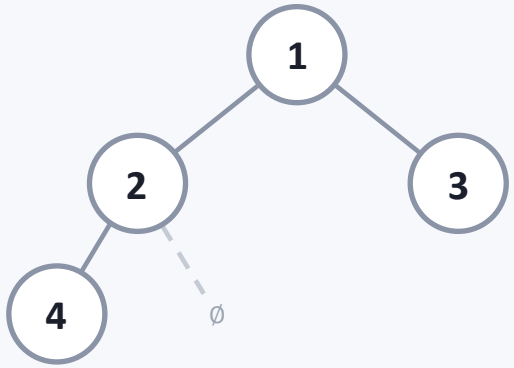


Every node has 0 or 2 children.

full ✓ complete ✗

Last level is not filled from the left (2 has no children, 3 does).

Complete

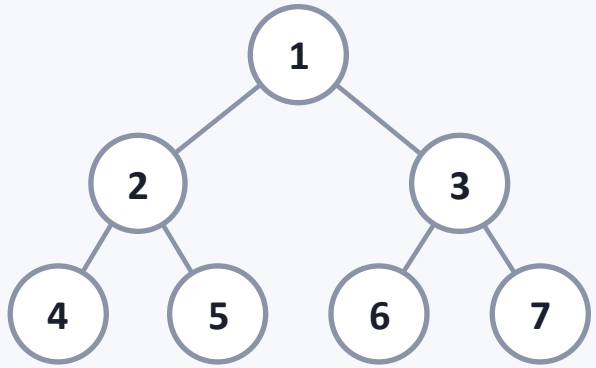


All levels full except possibly the last, which is filled left to right.

complete ✓ full ✗

Node 2 has exactly one child.

Perfect



All internal nodes have 2 children and all leaves have the same depth.

perfect ⇒ full ✓ and complete ✓

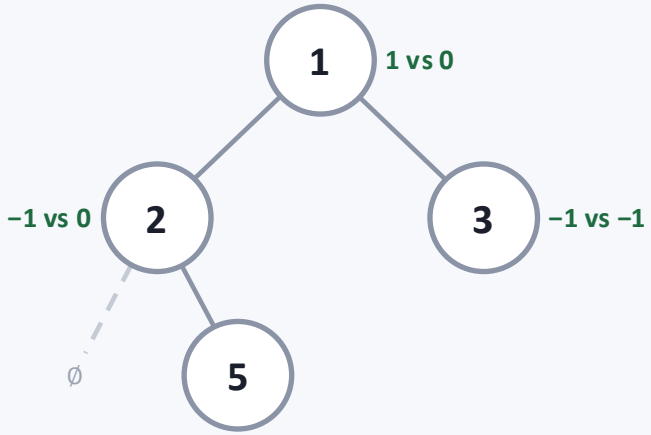
n = 2^(h+1) − 1 (here h = 2, n = 7)

Full and complete are independent: a tree can be either, both, or neither.

Height-balanced: the precise definition

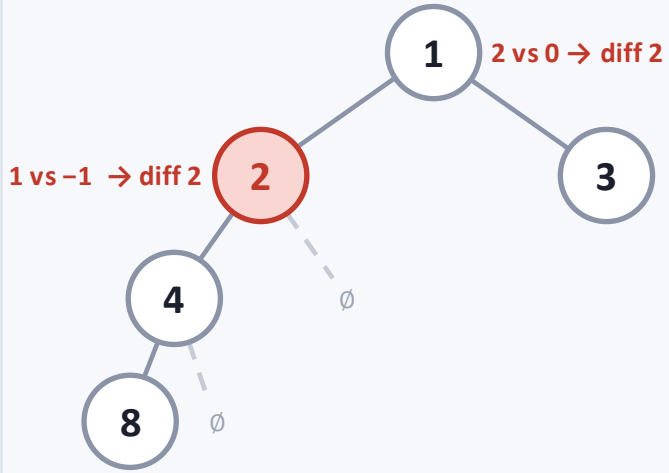
For EVERY node v: $|\text{height}(v.\text{left}) - \text{height}(v.\text{right})| \leq 1$

Balanced ✓ (but not complete)



labels: height(left) vs height(right)

Not balanced ✗



Fails at node 2 (and at the root).

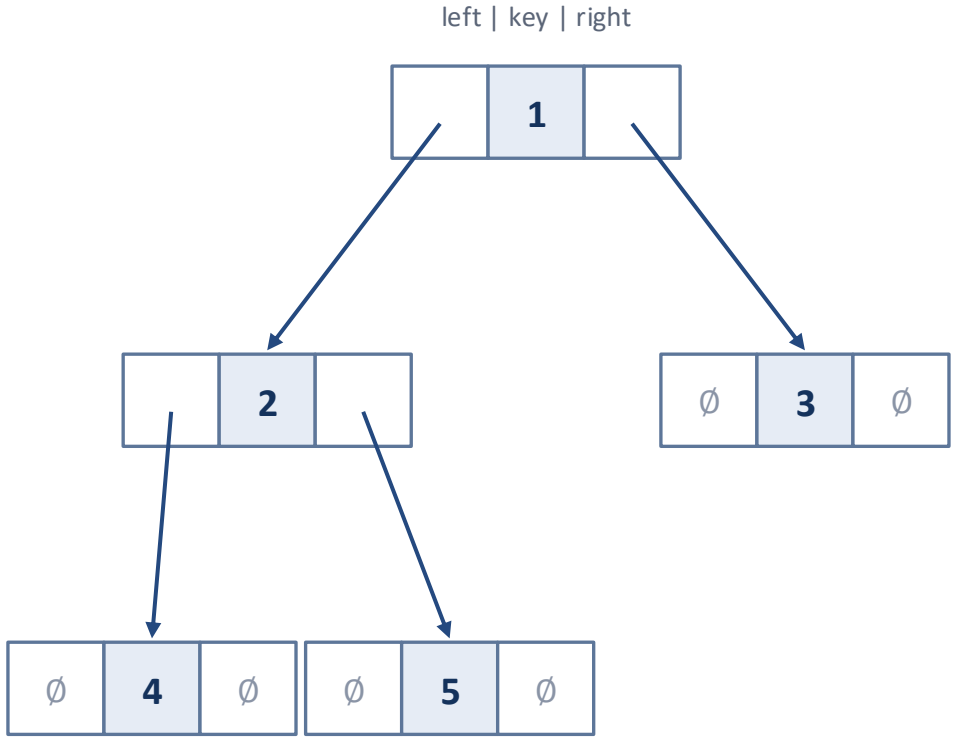
Why it matters: any binary tree with n nodes has $\lceil \log_2 n \rceil \leq h \leq n - 1$. Height-balanced trees keep h = O(log n) — the source of fast search (Section C).

Representing a binary tree in Java

```java
class Node {
    int key;
    Node left, right;    // null = empty subtree

    Node(int key) {
        this.key = key;  // left = right = null
    }
}

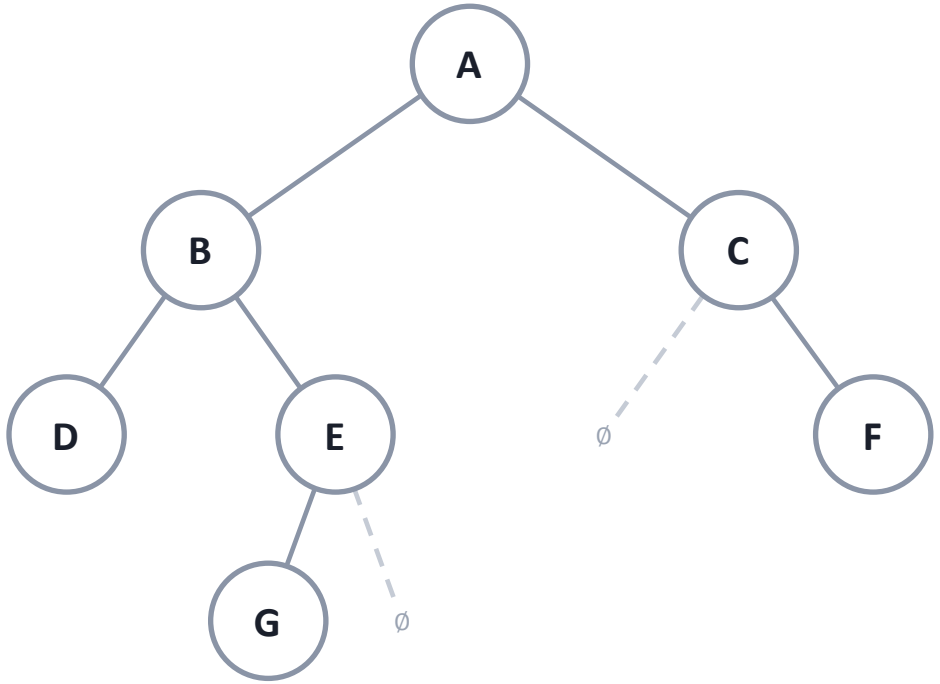
Node root = new Node(1);
root.left = new Node(2);
root.right = new Node(3);
root.left.left = new Node(4);
root.left.right = new Node(5);
```



Each node stores a key and two references; Ø = null. Space: Θ(n) nodes, 2 references each.

Tree Traversals

Three ways to walk the same tree



This tree is used for all three traversals.

Preorder

Root → Left → Right

visit the node before its subtrees

Inorder

Left → Root → Right

visit the node between its subtrees

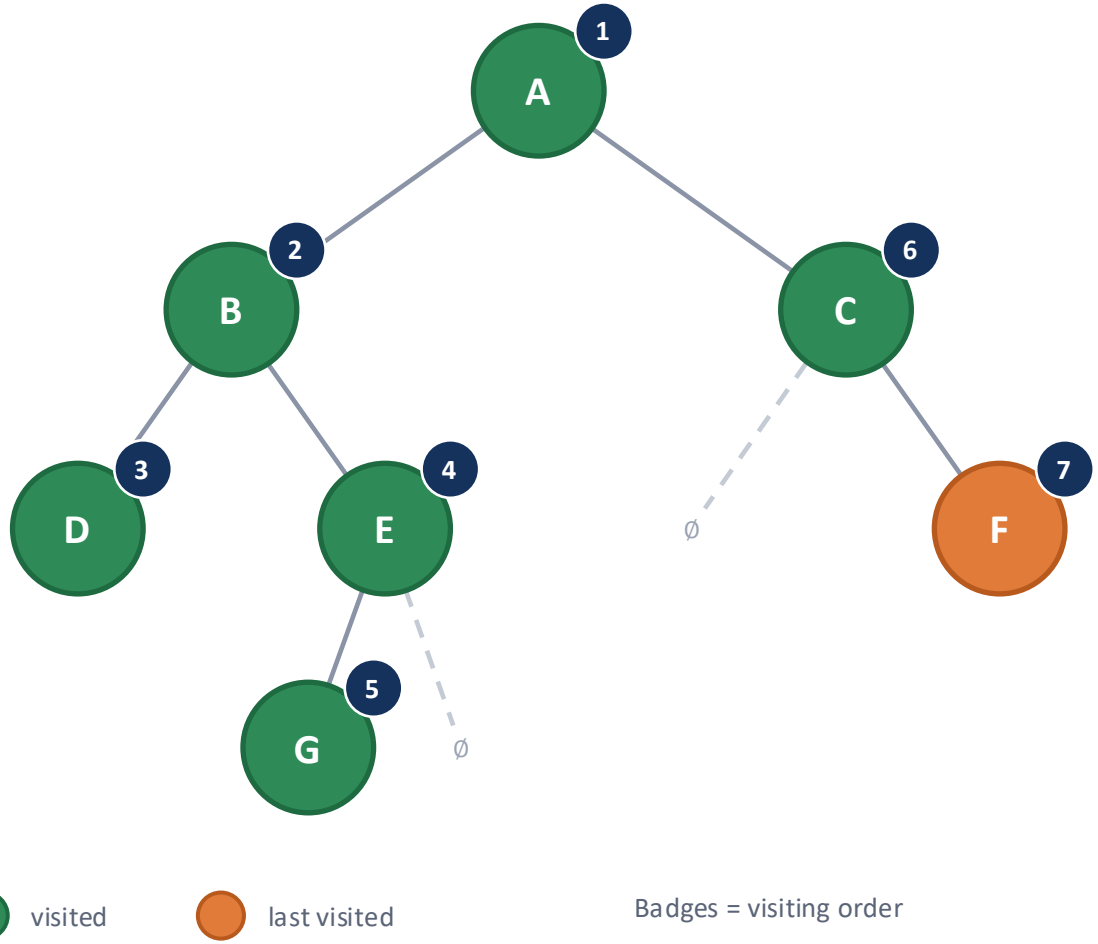
Postorder

Left → Right → Root

visit the node after its subtrees

Traversal = visit every node exactly once. The rule is applied recursively at every subtree.

Preorder trace: Root → Left → Right



Output

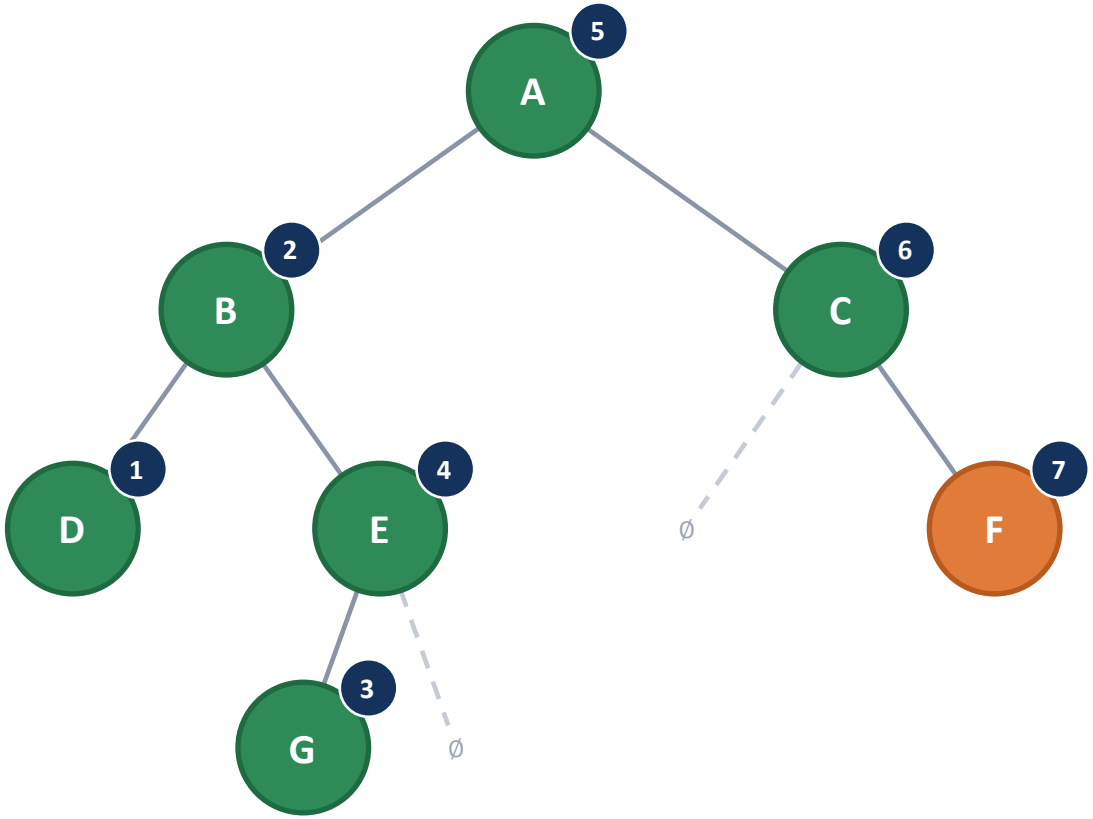
A	B	D	E	G	C	F
1	2	3	4	5	6	7

Step by step

At A: visit A → go left to B: visit B → left to D: visit D (no children) → back at B, go right to E: visit E → left to G: visit G → back up to A, go right to C: visit C → C has no left; right to F: visit F.

```
visit(v); walk(v.left); walk(v.right);
```

Inorder trace: Left → Root → Right



visited last visited Badges = visiting order

Output

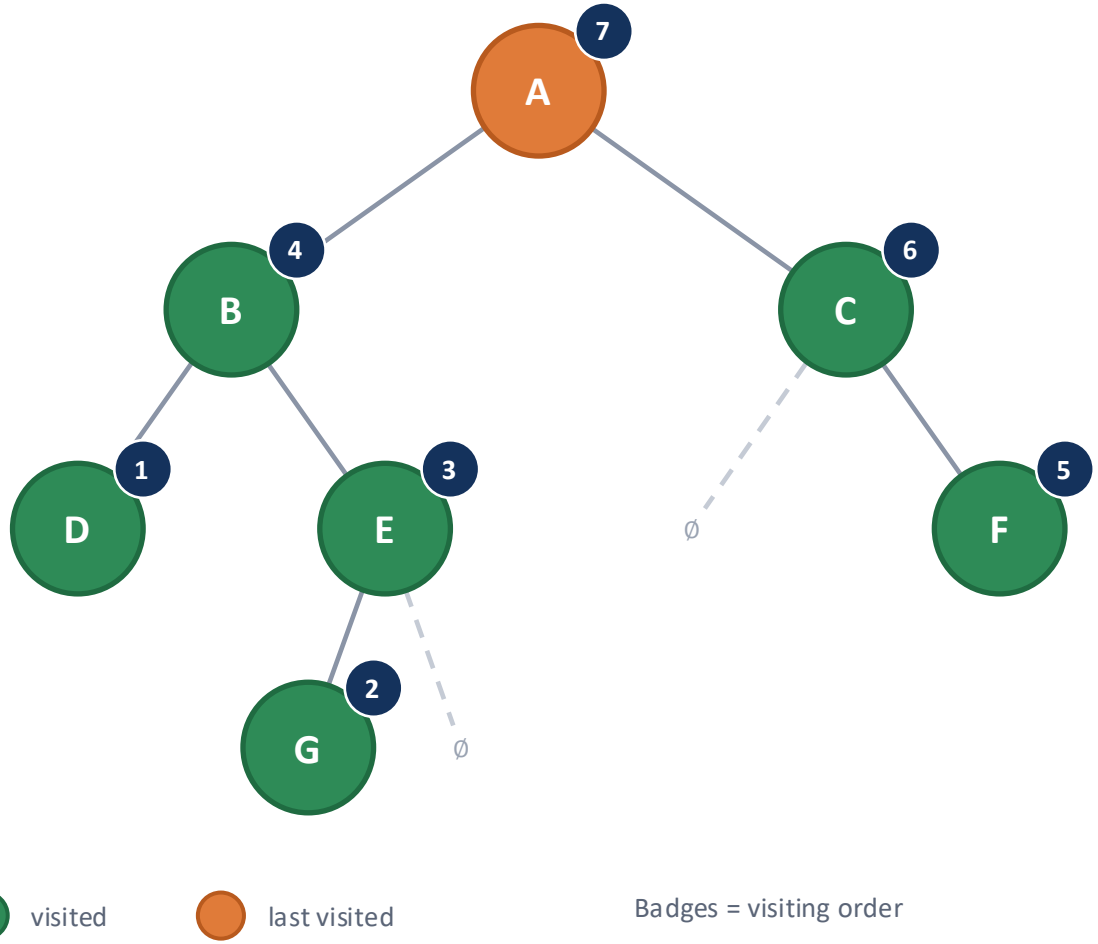
D	B	G	E	A	C	F
1	2	3	4	5	6	7

Step by step

At A: go left first → B: go left → D has no left: visit D → back at B: visit B → right to E: go left → G: visit G → back at E: visit E → left subtree of A finished: visit A → C has no left: visit C → right: visit F.

```
walk(v.left); visit(v); walk(v.right);
```

Postorder trace: Left → Right → Root



Output

D	G	E	B	F	C	A
1	2	3	4	5	6	7

Step by step

D has no children: visit D → B's right is E; E's left is G: visit G → E has no right: visit E → both subtrees of B done: visit B → A's right: C has no left; go to F: visit F → visit C → finally visit A.

```
walk(v.left); walk(v.right); visit(v);
```

Java: the three traversals differ in one line

```
1 void preorder(Node v) {
2     if (v == null) return;
3     visit(v);
4     preorder(v.left);
5     preorder(v.right);
6 }
```

```
1 void inorder(Node v) {
2     if (v == null) return;
3     inorder(v.left);
4     visit(v);
5     inorder(v.right);
6 }
```

```
1 void postorder(Node v) {
2     if (v == null) return;
3     postorder(v.left);
4     postorder(v.right);
5     visit(v);
6 }
```

Line 2 — base case: an empty subtree has nothing to visit (the Ø markers in the diagrams).

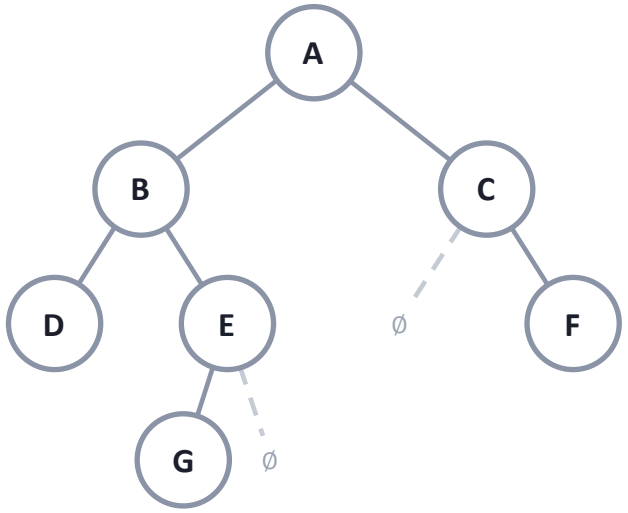
Highlighted line — visit: print the key, add it to a list, or do any O(1) work.

Two recursive calls: "the whole left subtree" and "the whole right subtree" from the rules.

Full classes: BinaryTree.java in the code package (returns the order as a List).

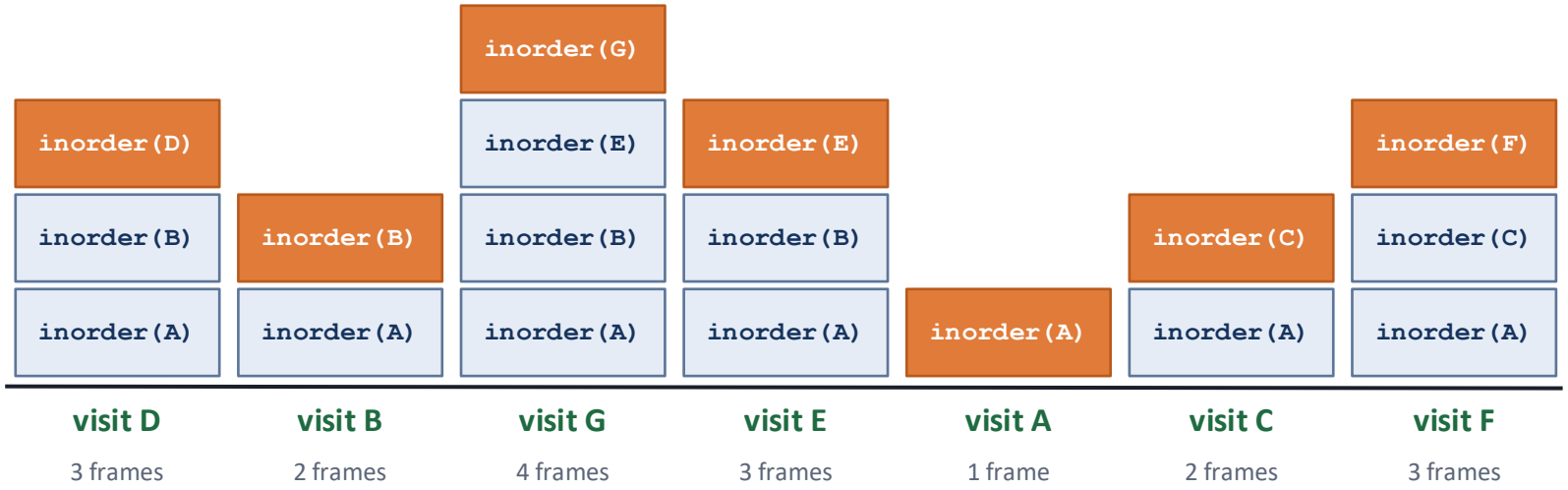
LO2 New

Inside the recursion: the call stack (inorder)



inorder(A) with the shared tree

Stack snapshots at the moment each node is visited (top = running frame)



The stack holds exactly the path from the root to the current node. Deepest point: 4 frames at G (depth 3) → stack depth ≤ h + 1 frames (+1 for a call on an empty child).

LO2 LO4 New

Cost of a traversal: O(n) time, O(h) extra space

Time: Θ(n)

Each node is visited exactly once; each call does O(1) work besides its two recursive calls.

Calls = n (real nodes) + (n + 1) (null children) = 2n + 1

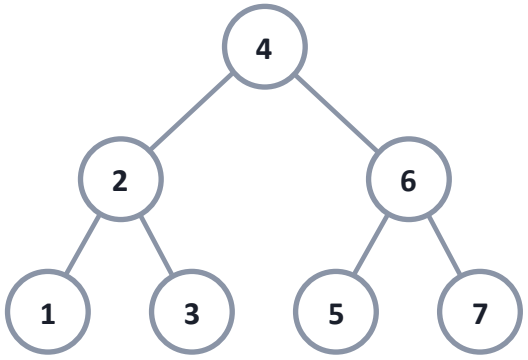
⇒ T(n) = Θ(n), for every shape of tree.

Auxiliary space: Θ(h)

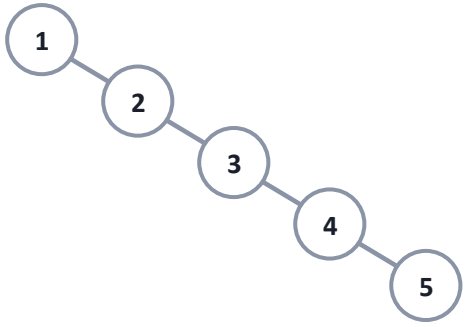
Stack holds the current root-to-node path: at most h + 2 frames.

Balanced: h = Θ(log n) → Θ(log n) space

Skewed chain: h = n − 1 → Θ(n) space



n = 7, h = 2 → ≤ 4 frames



chain: n = 5, h = 4 → ≤ 6 frames

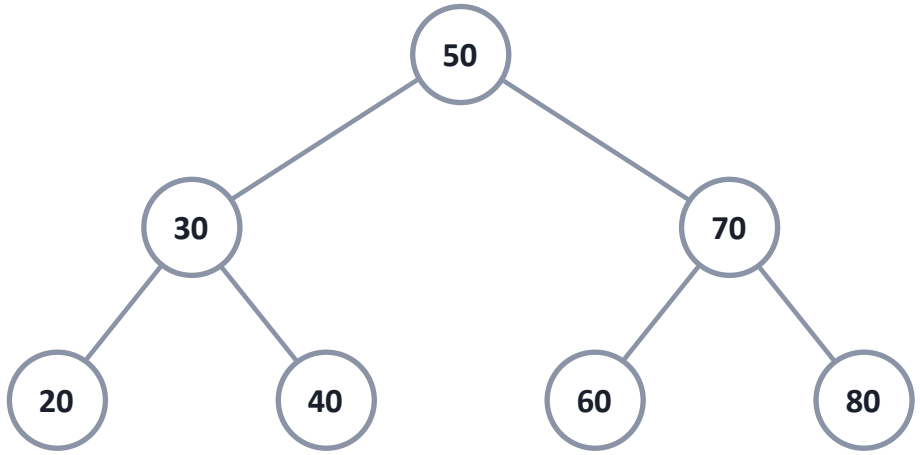
Binary Search Trees

The BST invariant: whole subtrees, not just children

For EVERY node x: all keys in x.left < x.key < all keys in x.right

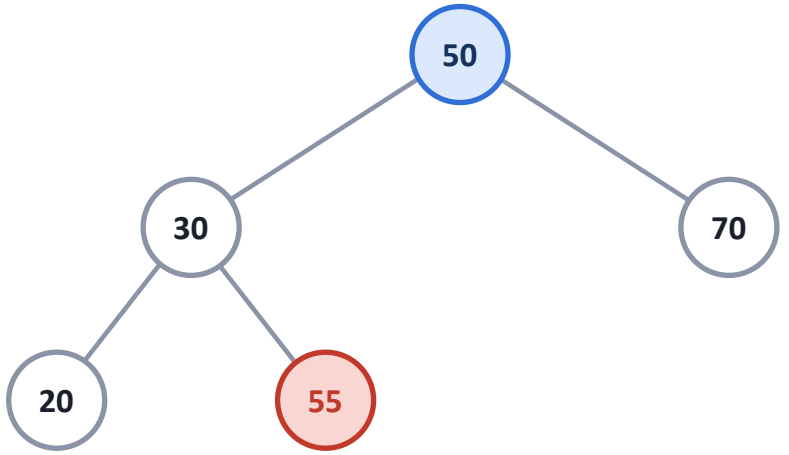
For now: all keys are distinct (duplicates: slide 27).

Valid BST ✓



Every key in 50's left subtree (20, 30, 40) is < 50; every key in its right subtree (60, 70, 80) is > 50. Same at 30 and 70.

NOT a BST ✗ — though every parent–child pair looks fine



55 > 30, so it may be 30's right child… but 55 is in 50's LEFT subtree and 55 > 50. A search for 55 would go right at 50 and miss it.

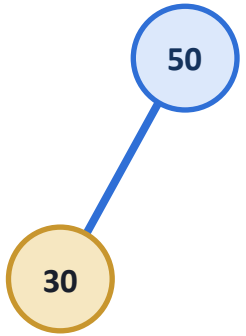
Building a BST: insert 50, 30, 70, 20

insert 50



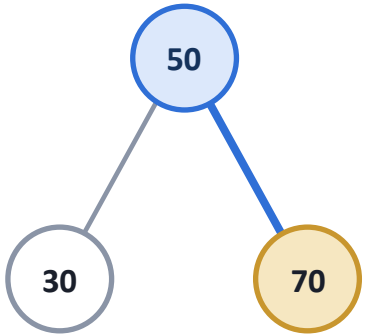
Empty tree: 50 becomes the root.

insert 30



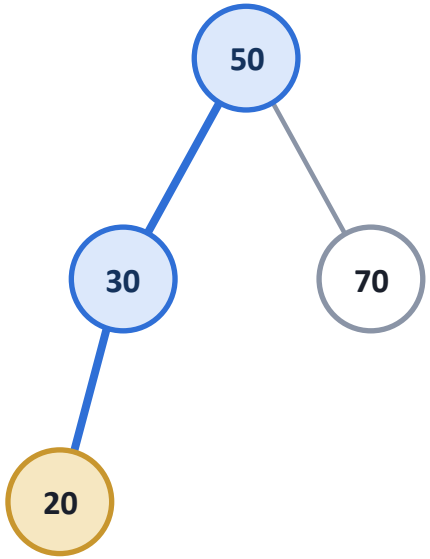
30 < 50 → go left → empty: attach.

insert 70



70 > 50 → go right → empty: attach.

insert 20



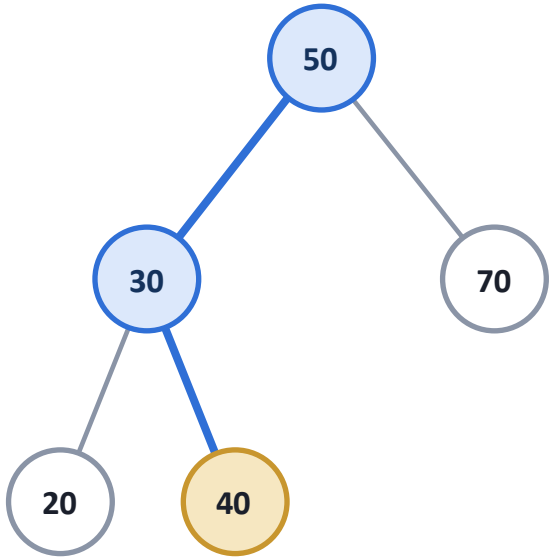
20 < 50 → left; 20 < 30 → left → attach.

compared newly inserted leaf

Sequence: 50, 30, 70, 20, 40, 60, 80

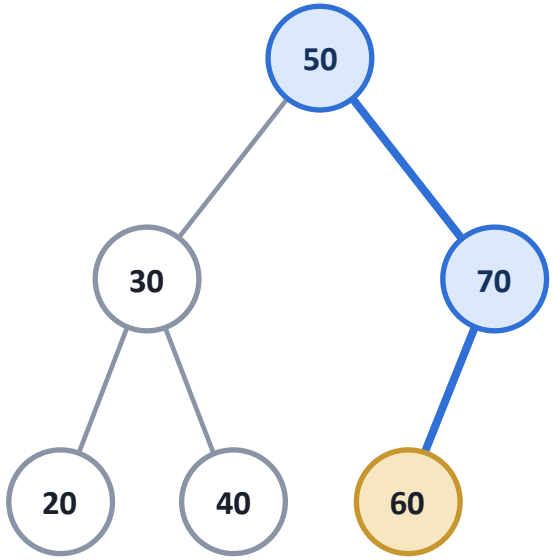
Building a BST: insert 40, 60, 80

insert 40



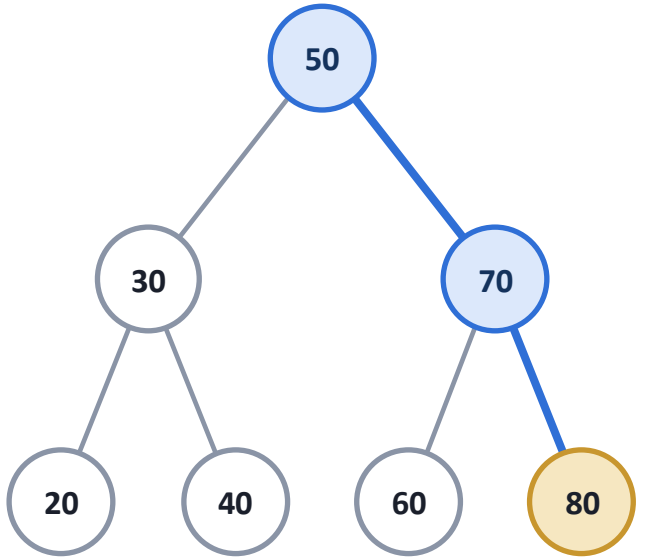
40 < 50 → left; 40 > 30 → right → attach.

insert 60



60 > 50 → right; 60 < 70 → left → attach.

insert 80



80 > 50 → right; 80 > 70 → right → attach.

compared

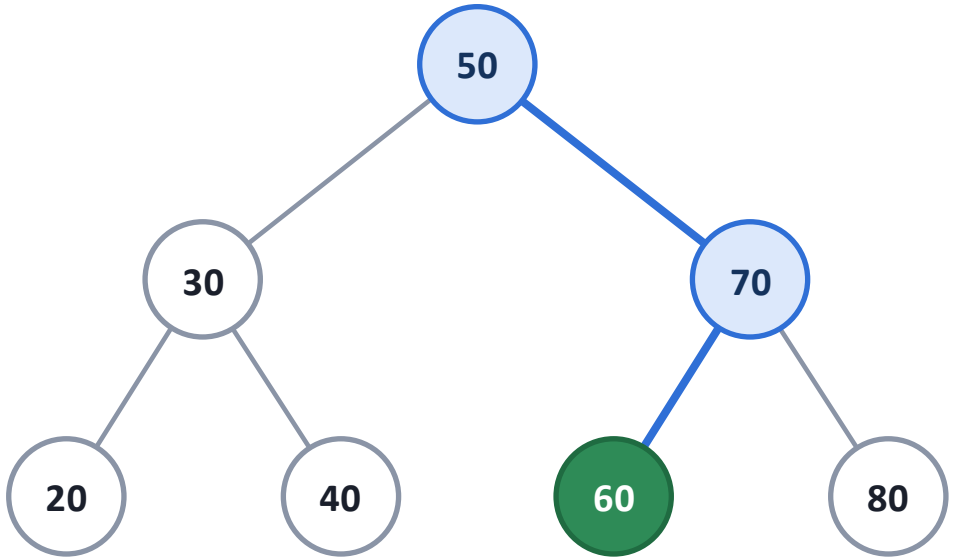
newly inserted leaf

Insertion = unsuccessful search + attach a new leaf where the search fell off.

LO3 LO4 New

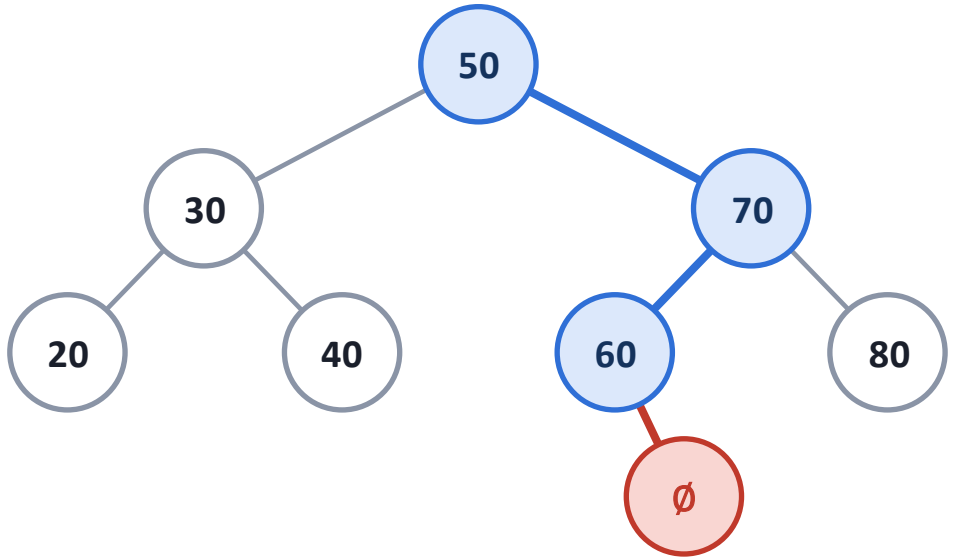
Search: one comparison per level

search(60) — hit



60 > 50 → right · 60 < 70 → left · 60 = 60 → found
3 comparisons

search(65) — miss



65 > 50 → right · 65 < 70 → left · 65 > 60 → right: null → not found
3 comparisons

Each comparison discards an entire subtree. Cost = number of nodes on the path ≤ h + 1 ⇒ O(h).

Java: search (iterative) and insert (recursive)

```java
boolean contains(int key) {
    Node cur = root;
    while (cur != null) {
        if (key == cur.key) return true;
        if (key < cur.key) cur = cur.left;
        else               cur = cur.right;
    }
    return false;   // fell off the tree
}
```

```java
// call as: root = insert(root, key);
Node insert(Node node, int key) {
    if (node == null)               // empty spot
        return new Node(key);
    if (key < node.key)
        node.left = insert(node.left, key);
    else if (key > node.key)
        node.right = insert(node.right, key);
    // key == node.key: duplicate, ignore
    return node;    // re-link this subtree
}
```

Reading the search

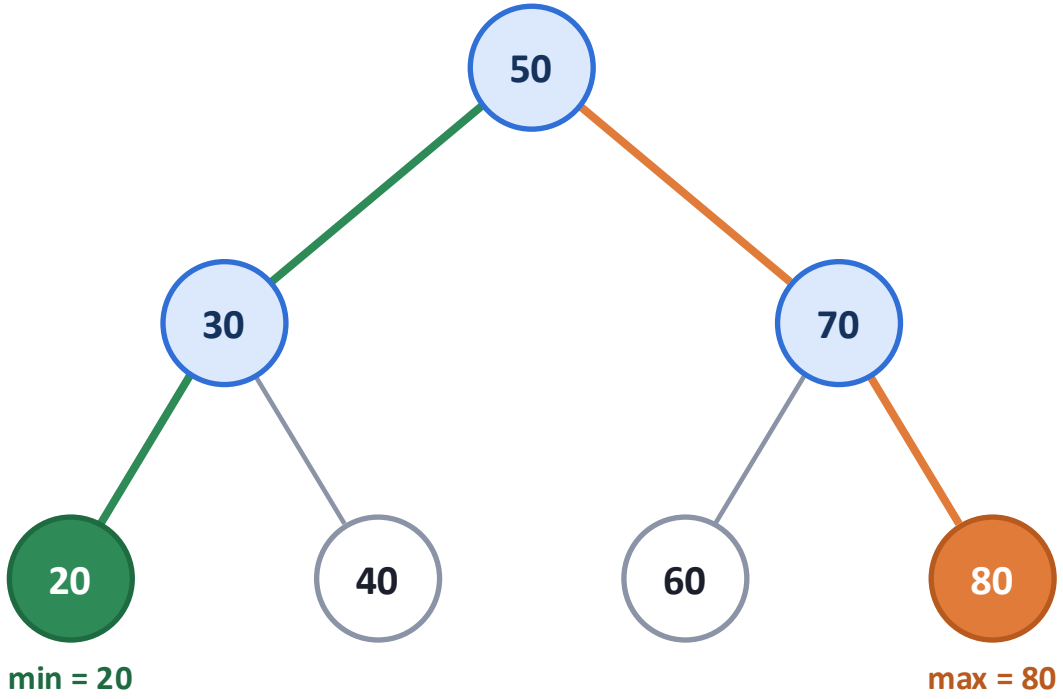
Highlighted lines = the blue path on the previous slide: move left or right, discarding the other subtree. The loop runs at most h + 1 times.

Reading the insert

Same path, recursively. At null we create the leaf; on the way back each call returns its (possibly new) subtree root, so the parent's link is updated. Duplicate policy: reject.

LO3 New

Minimum and maximum: keep going left / right




```
1  int min() {            // non-empty tree
2      Node cur = root;
3      while (cur.left != null) cur = cur.left;
4      return cur.key;
5  }
6  int max() {
7      Node cur = root;
8      while (cur.right != null) cur = cur.right;
9      return cur.key;
10 }
```

Why it works: every key smaller than x is in x.left, so the smallest key is the node with no left child on the leftmost path. **Also needed for deletion:** successor(x) = min of x.right.

Duplicate keys: choose a policy and apply it everywhere

1 · Reject (set semantics)

insert(40) when 40 exists → ignore (or update the value in a map).

Default in this course and in java.util.TreeMap.

2 · Count field

Each node stores key + count; a duplicate increments count.

Shape unchanged; delete decrements count first.

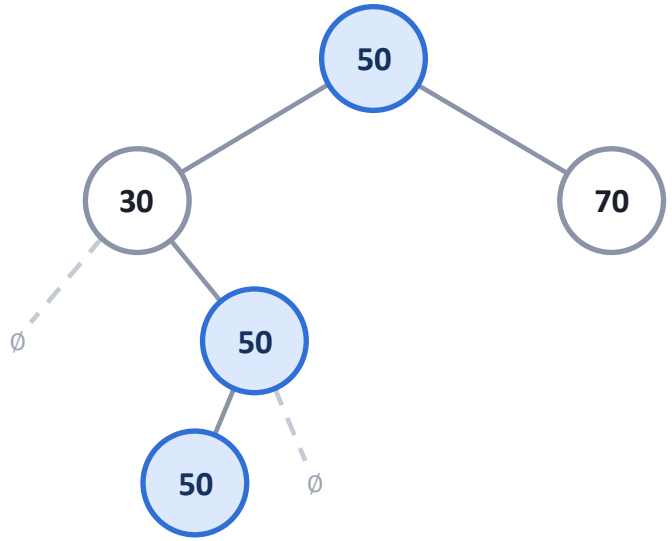
3 · Consistent side

Equal keys go left: invariant becomes left ≤ x < right.

search, delete and range queries must all use ≤ on the left.

Never mix policies: insert with ≤ but search with < loses keys.

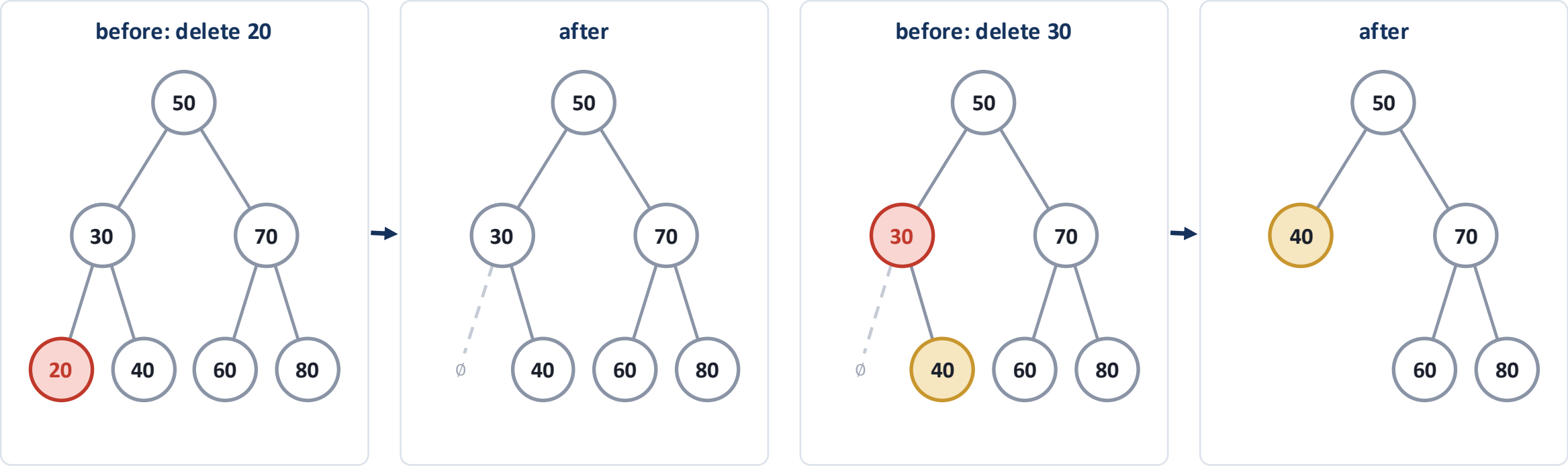
Policy 3: insert 50, 30, 50, 70, 50



The copies of 50 are NOT next to each other: the 2nd went to 30's right. Finding all 50s means searching the left subtree too. Many duplicates → long chains.

LO3 New

Deletion, cases 1 and 2: leaf, one child



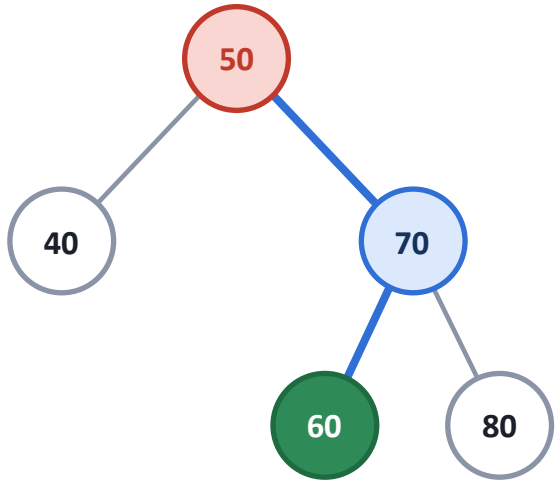
Case 1 · leaf: set the parent's reference to null. Nothing else changes.

Case 2 · one child: the parent adopts the only child (40 takes 30's place). All of 30's subtree was < 50, so the invariant still holds.

LO3 New

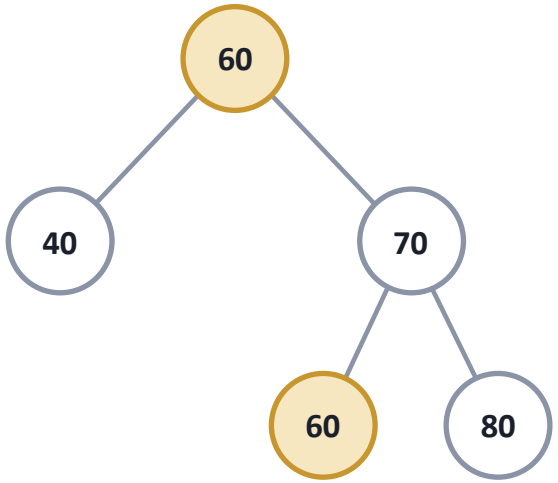
Deletion, case 3: two children → inorder successor

① find successor = min(right)



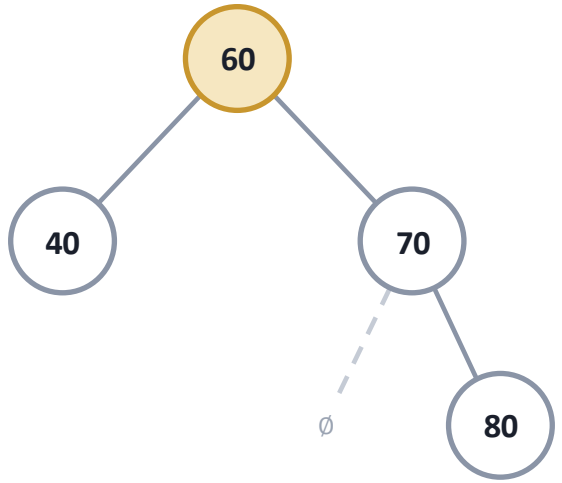
go right once (70), then left until no left child: 60

② copy 60 into the node



now 60 appears twice

③ delete old 60 from right subtree



successor has no left child → case 1 or 2

Why 60? It is larger than everything in the left subtree (40) and smaller than everything else in the right subtree (70, 80) — so it can sit where 50 was. (Inorder predecessor = max(left) also works.)

LO3 New

Java: deletion handles all three cases

```
1  // call as: root = delete(root, key);
2  Node delete(Node node, int key) {
3      if (node == null) return null;          // not found
4      if (key < node.key) {
5          node.left = delete(node.left, key);
6      } else if (key > node.key) {
7          node.right = delete(node.right, key);
8      } else {                                // found it
9          if (node.left == null) return node.right;
10         if (node.right == null) return node.left;
11         Node succ = node.right;             // case 3
12         while (succ.left != null) succ = succ.left;
13         node.key = succ.key;
14         node.right = delete(node.right, succ.key);
15     }
16     return node;
17 }
```

Lines 3–7

search for the key, re-linking on the way back (as in insert)

Line 9

no left child: return the right subtree. Covers case 1 (leaf → returns null) and case 2 (only a right child)

Line 10

no right child: case 2 (only a left child)

Lines 11–14

case 3: find min of right subtree, copy its key, delete it from the right subtree

Cost: O(h) to find + O(h) to find the successor + O(h) to remove it = O(h).

LO4 New

Every BST operation costs O(h)

Derivation

search, insert, delete, min, max each follow ONE path from the root downward.

Each level: O(1) work (one comparison + one move).

A path visits at most h + 1 nodes.

⇒ T(n) = c · (h + 1) = O(h)

How large is h?

$$\lfloor \log_2 n \rfloor \leq h \leq n - 1$$

Balanced: h = Θ(log n) → O(log n)

Skewed (chain): h = n − 1 → O(n)

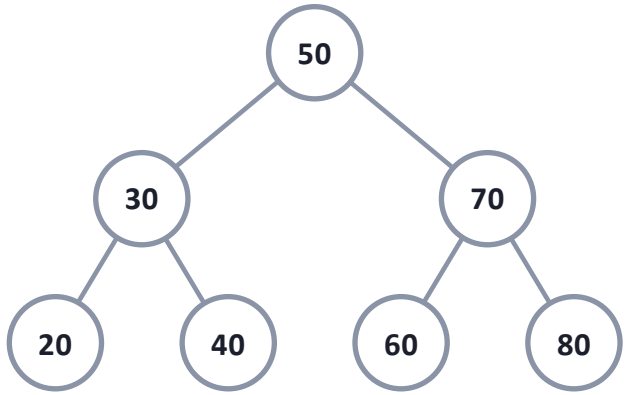
Random insertion order: expected h = O(log n)

n keys	balanced: ≈ $\log_2 n$ comparisons	chain: up to n
1,000	≈ 10	1,000
1,000,000	≈ 20	1,000,000
1,000,000,000	≈ 30	1,000,000,000

LO4 New

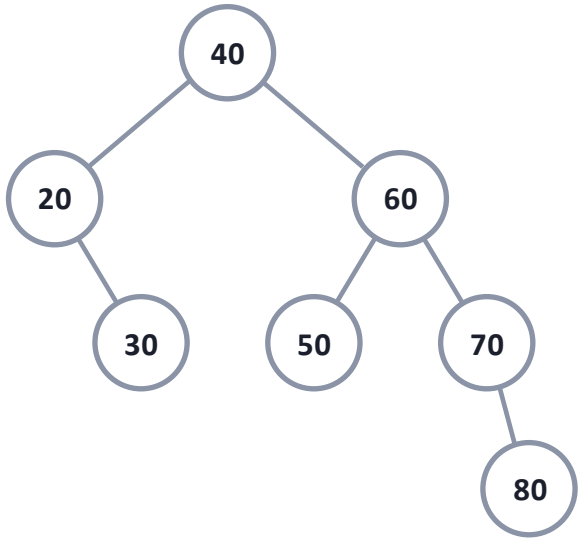
Same 7 keys, different insertion order, different cost

50, 30, 70, 20, 40, 60, 80



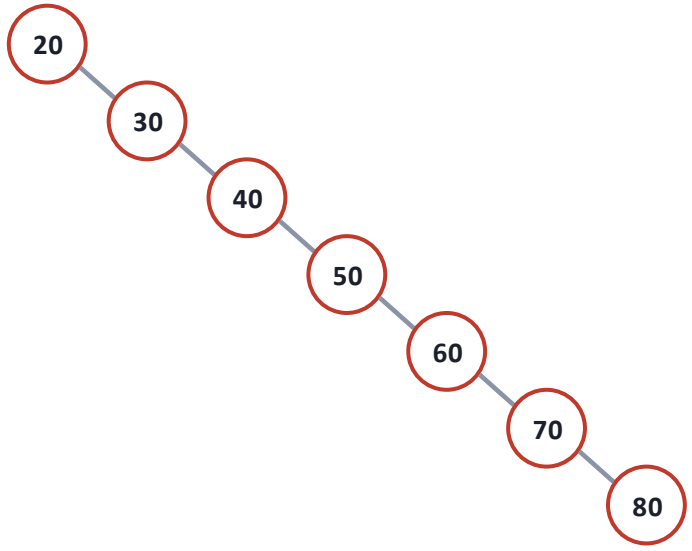
h = 2 (best possible)

40, 20, 60, 30, 50, 70, 80



h = 3

20, 30, 40, 50, 60, 70, 80 (sorted!)



h = 6 = n − 1 (a linked list!)

An ordinary BST never rebalances itself. Self-balancing BSTs — AVL trees, red-black trees (java.util.TreeMap) — restore balance with rotations after insert/delete, guaranteeing h = O(log n). Covered later; no rotations today.

Hash Tables and Separate Chaining

LO5 LO8 New

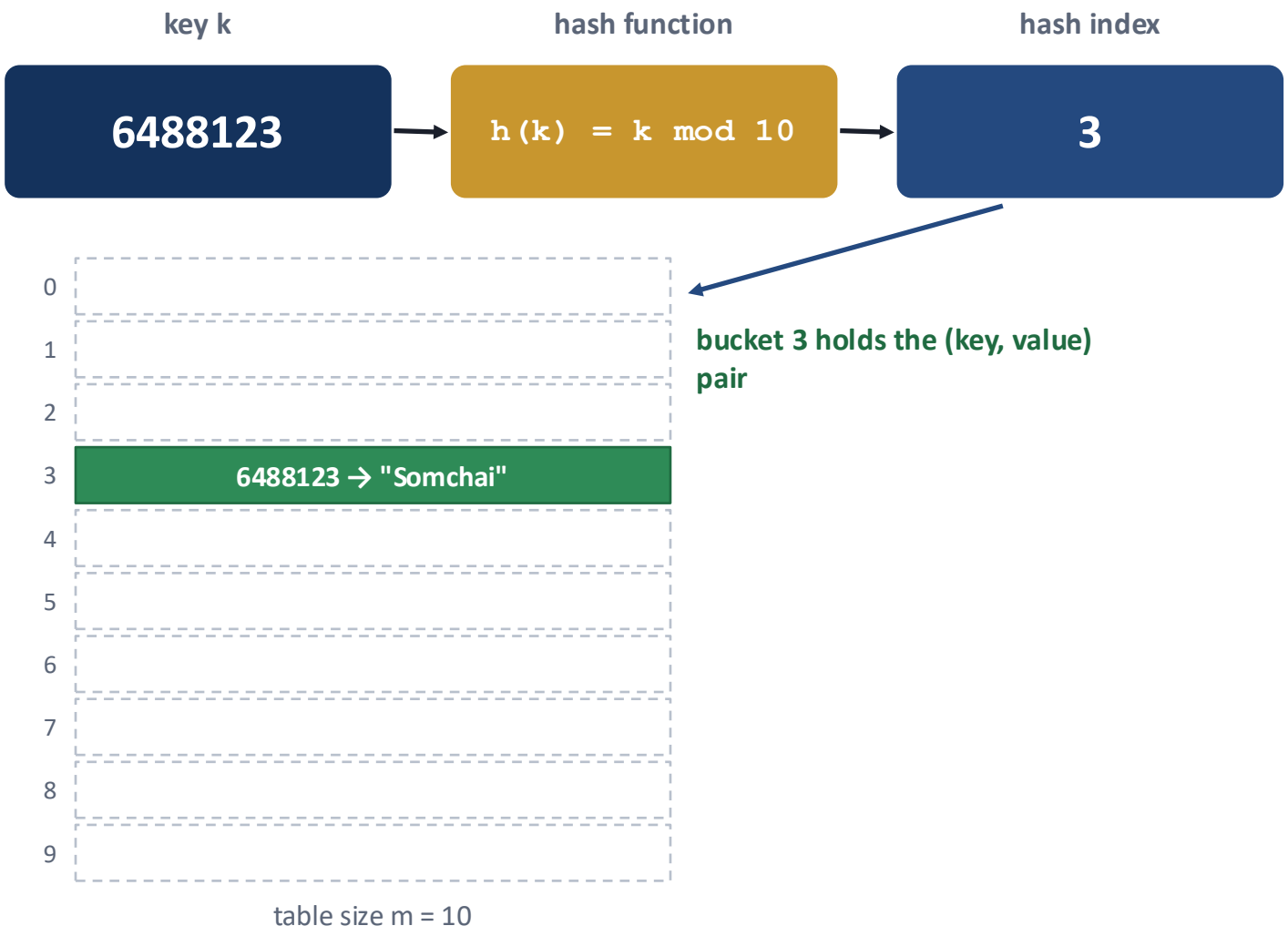
The lookup problem: find a student record by ID

30,000 students, 7-digit IDs (0000000 – 9999999). Operations: insert, find, delete by ID.

Structure	find	insert	catch
Unsorted array / list	**O(n)**	O(1)	scan everything
Sorted array + binary search	**O(log n)**	O(n)	shifting on insert
Balanced BST	**O(log n)**	O(log n)	≈ 15 comparisons
Direct addressing: table[id]	**O(1)**	O(1)	10,000,000 slots for 30,000 keys (0.3 % used)
Hash table: table[h(id)], m ≈ n	**O(1) expected**	O(1) expected	small table; collisions must be handled

Idea: keep direct addressing's "compute the position" trick, but squeeze the huge key range into a small table with a hash function $h(k) \in \{0, \ldots, m-1\}$.

Hash table vocabulary



Key: what we search by (student ID)

Value: data associated with the key (record)

Bucket / slot: one position of the array, 0 … m−1

Hash function h: maps every key to an index in [0, m)

Hash index: h(k): where k should go

Collision: $h(k_1) = h(k_2)$ for different keys

Load factor α: n / m: average keys per bucket

Distribution: how evenly h spreads real keys

LO5 LO7 Revised: s.19, 25–27

Good hash functions — and two Java pitfalls

Requirements

Deterministic: same key → same index, always

Uniform: spreads real keys evenly

Fast: O(1) to compute

Pitfall 1: negative keys

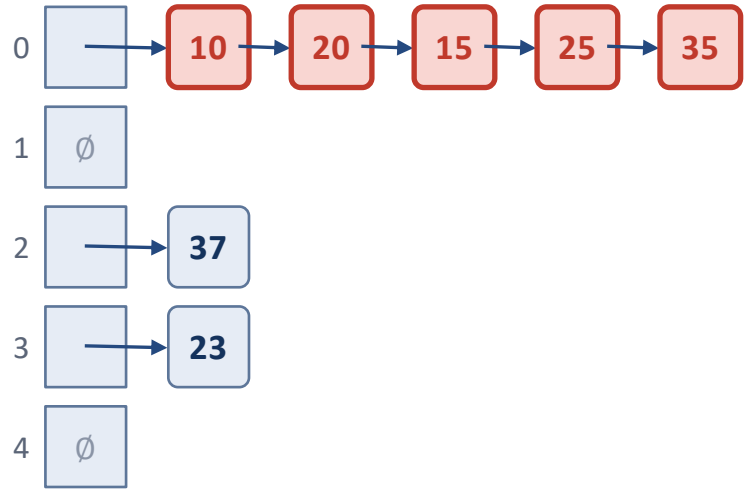
`-13 % 10` → `-3` (index out of bounds!)

`Math.floorMod(-13, 10)` → `7`

Java's % keeps the sign of the dividend.

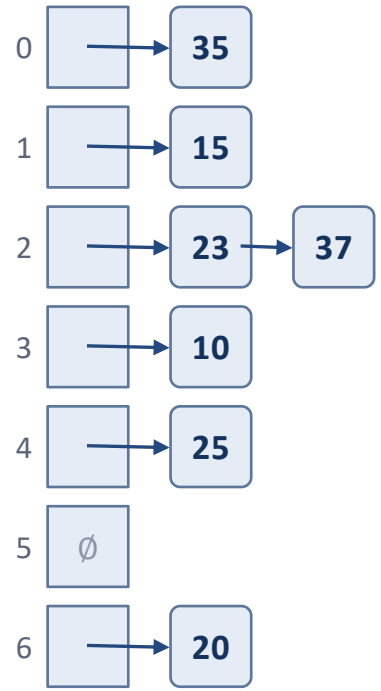
Pitfall 2: table size vs. patterns in the keys (keys 10, 20, 23, 15, 25, 37, 35)

m = 5 (original slide 26)



5 of 7 keys in one bucket: every key ending in 0 or 5 shares a factor with m = 5.

m = 7 (prime)

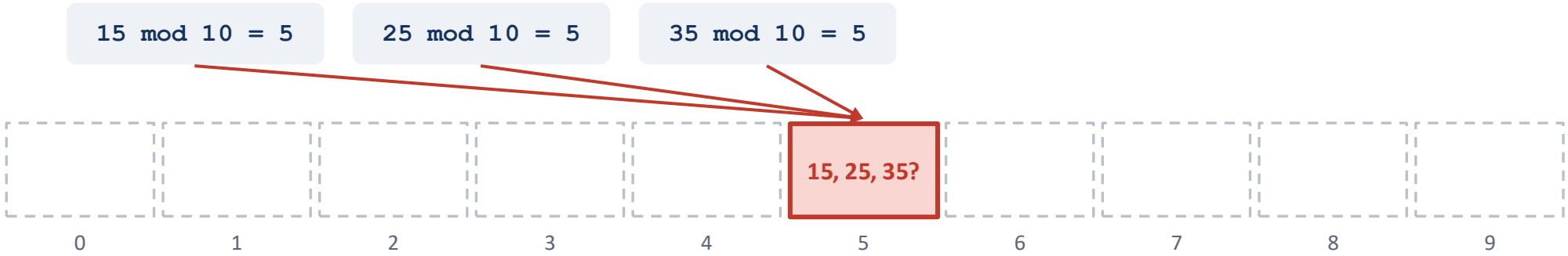


Longest chain = 2. A prime m not close to a power of 10 breaks such patterns.

Collisions are unavoidable

m = 10, h(k) = k mod 10, keys 15, 25, 35

So every hash table needs a collision-resolution strategy.



Pigeonhole principle

If the universe U has more possible keys than there are slots (|U| > m), then some two keys MUST share an index — for any hash function. Here |U| = all ints, m = 10.

Birthday paradox

Collisions appear long before the table is full: with m = 365 buckets and random hashing, 23 keys already give a > 50 % chance that two collide.

LO5 LO8 New

Load factor α = n / m controls performance

$$\alpha = n / m = \text{(keys stored)} / \text{(number of buckets)}$$

n = 3, m = 10 → α = 0.3



n = 7, m = 10 → α = 0.7



Expected vs. worst case

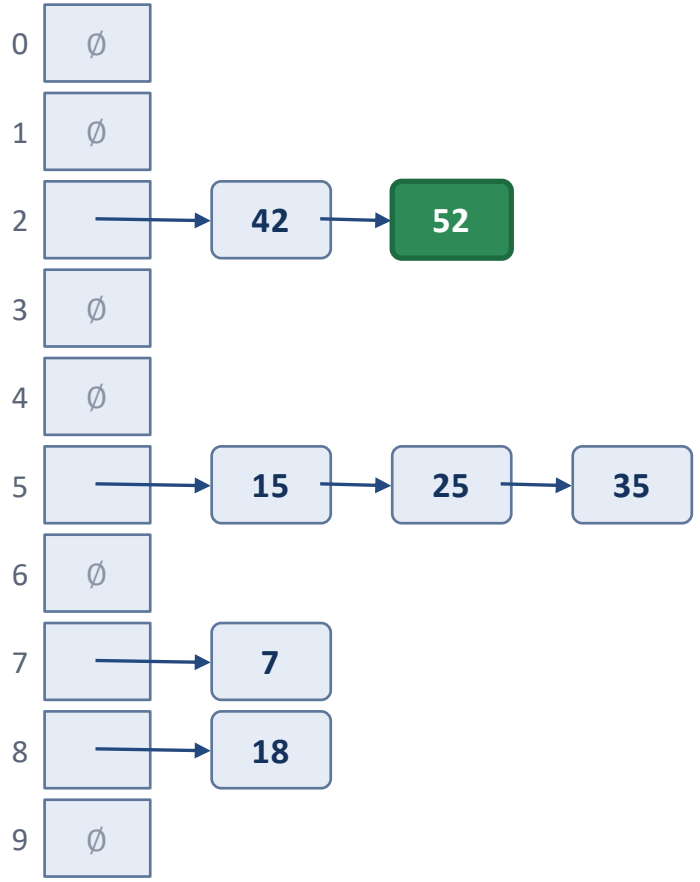
Expected (average over random hashing): O(1 + α). If we resize to keep α ≤ a constant, this is O(1).

Worst case: all n keys hash to one bucket → Θ(n). Hashing never guarantees O(1) worst-case lookup.

Higher α → more collisions → longer searches. Real tables rehash into a larger array (typically 2m) when α passes a threshold — java.util.HashMap's default threshold is 0.75. Each rehash is O(n), but doubling makes the cost O(1) amortized per insert.

Separate chaining: each bucket holds a list

m = 10, h(k) = k mod 10. Insert 15, 25, 35, 42, 7, 18, 52

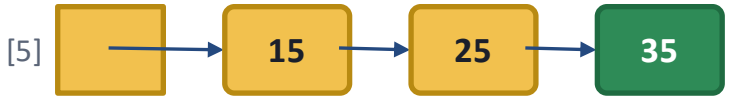


key	k mod 10	
15	5	
25	5	collision → append
35	5	collision → append
42	2	
7	7	
18	8	
52	2	collision → append

Collisions are no problem: the key joins its bucket's list. n = 7, m = 10 → α = 0.7; longest chain = 3.

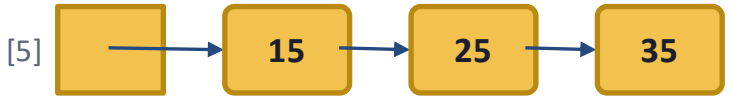
Chaining: search and delete scan one chain only

`search(35)`



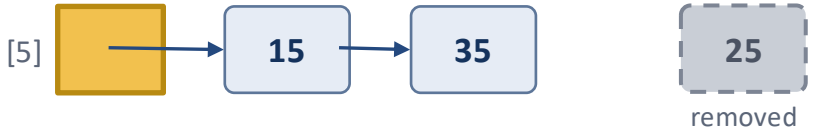
bucket 5: compare 15, 25, 35 → found (3 comparisons)

`search(45)`



bucket 5: compare 15, 25, 35 → end of list → not found (3 comparisons)

`delete(25)`



find 25 in bucket 5, unlink it: 15 → 35. No other bucket is touched.

compared found removed

Java: separate chaining

```java
LinkedList<Entry>[] buckets;   // m chains

int hash(int key) {
    return Math.floorMod(key, buckets.length);
}

void put(int key, String value) {
    LinkedList<Entry> chain = buckets[hash(key)];
    for (Entry e : chain) {
        if (e.key == key) {       // duplicate key:
            e.value = value;      // update value
            return;
        }
    }
    chain.add(new Entry(key, value));  // append
}
```

```java
String get(int key) {
    for (Entry e : buckets[hash(key)]) {
        if (e.key == key) return e.value;
    }
    return null;                  // not in chain
}

boolean remove(int key) {
    LinkedList<Entry> chain = buckets[hash(key)];
    Iterator<Entry> it = chain.iterator();
    while (it.hasNext()) {
        if (it.next().key == key) {
            it.remove();          // unlink node
            return true;
        }
    }
    return false;
}
```

Corrections vs. original: floorMod for negative keys · duplicate check before add · single pass in remove. (The original's remove((Integer) key) cast was needed: remove(int) would remove by INDEX.)

LO5 LO6 LO8 New

Chaining cost: expected Θ(1 + α), worst case Θ(n)

Expected (good hash)

Simple uniform hashing: every key equally likely in any bucket. Expected chain length = α.

Unsuccessful search: 1 + α

Successful search: ≈ 1 + α/2

→ O(1) if α is kept constant.

Worst case

All n keys in one bucket (bad h or adversarial keys, e.g. m = 5 with multiples of 5).

Search / insert / delete: Θ(n) — no better than a list.

Better buckets

Replace a long list by a balanced BST: worst case per bucket Θ(log n). Java 8+ HashMap does this when a bucket reaches 8 entries.

Space: Θ(m + n) — the array plus one list node per key. **Load factor can exceed 1** (chains just get longer), but performance degrades linearly in α.

Open Addressing

Open addressing: everything lives inside the array

On a collision, try other slots in a fixed order — the key's probe sequence:

$$h(k, 0),\ h(k, 1),\ h(k, 2),\ \ldots,\ h(k, m-1)$$

Linear probing

$h(k, i) = (h(k) + i) \bmod m$

step 1, 1, 1, …

Quadratic probing

$h(k, i) = (h(k) + i^2) \bmod m$

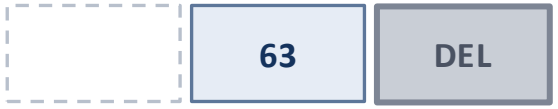
offsets 0, 1, 4, 9, 16, …

Double hashing

$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

step $h_2(k)$ depends on the key

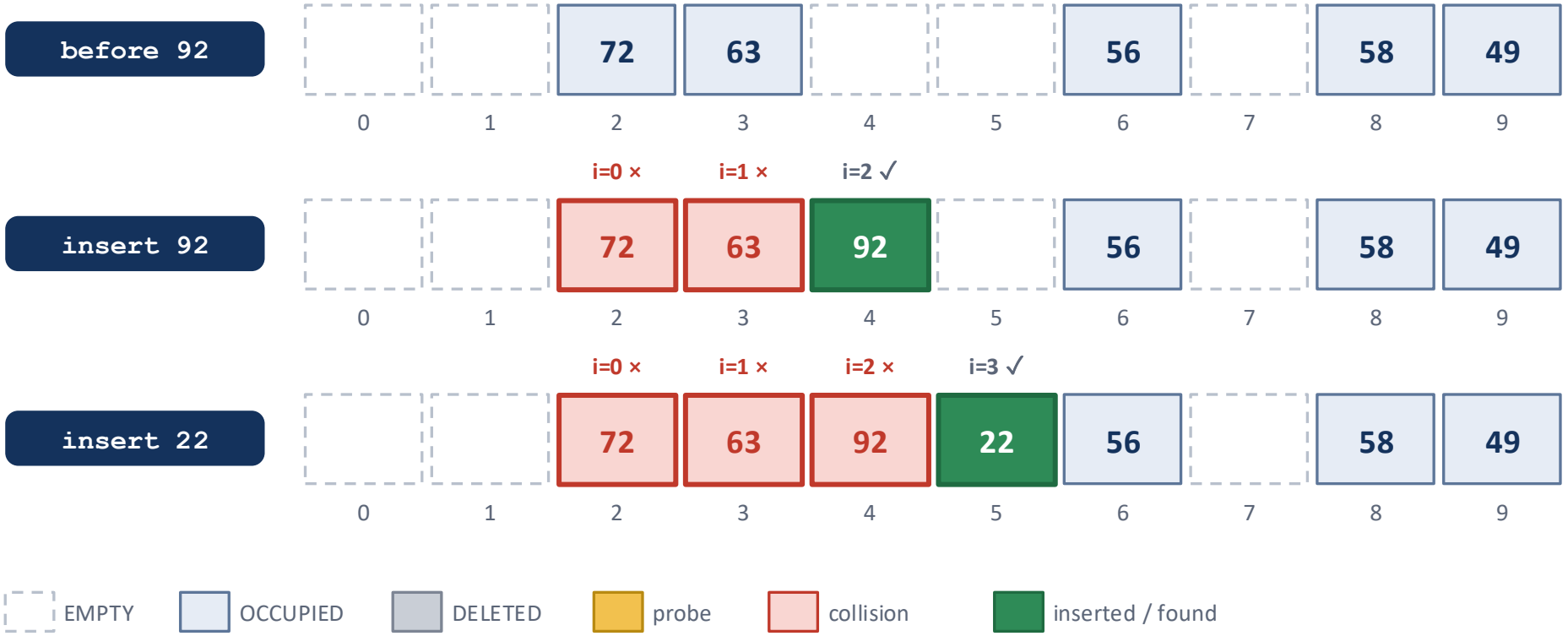
Each slot has a STATE (not a magic key value):



EMPTY never used → a search can stop here. **OCCUPIED** holds a key. **DELETED** held a key that was removed ("tombstone"). Always $\alpha = n/m \le 1$.

Linear probing: insert 49, 56, 72, 63, 92, 58, 22

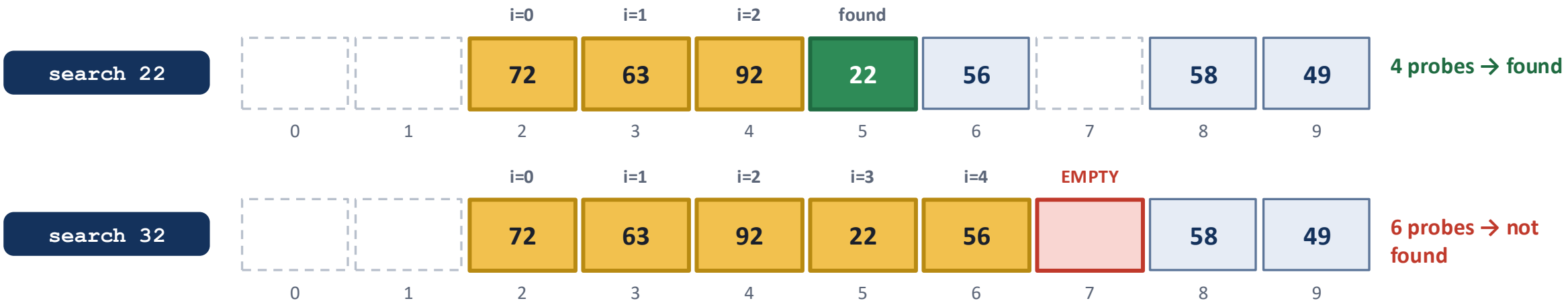
$$m = 10,\ h(k, i) = (k \bmod 10 + i) \bmod 10$$



key	h(k)	probes	slot
49	9	9	9
56	6	6	6
72	2	2	2
63	3	3	3
92	2	2, 3, 4	4
58	8	8	8
22	2	2, 3, 4, 5	5

Correction: only 92 and 22 collide. The original comments "collision occurs here" on 63 and 58 were wrong (their home slots were free).

Linear probing search: stop at the key or at EMPTY



Why stopping at EMPTY is correct

If 32 were in the table, its insertion would have walked the same sequence 2, 3, 4, … and taken the first free slot — slot 7 at the latest. Slot 7 is still EMPTY, so 32 was never inserted.

Bound the loop by m probes

Original: while (table[index] != -1) … — in a FULL table no slot is EMPTY, so searching for an absent key loops forever. Fix: for (i = 0; i < m; i++).

The deletion bug: marking a slot EMPTY breaks search

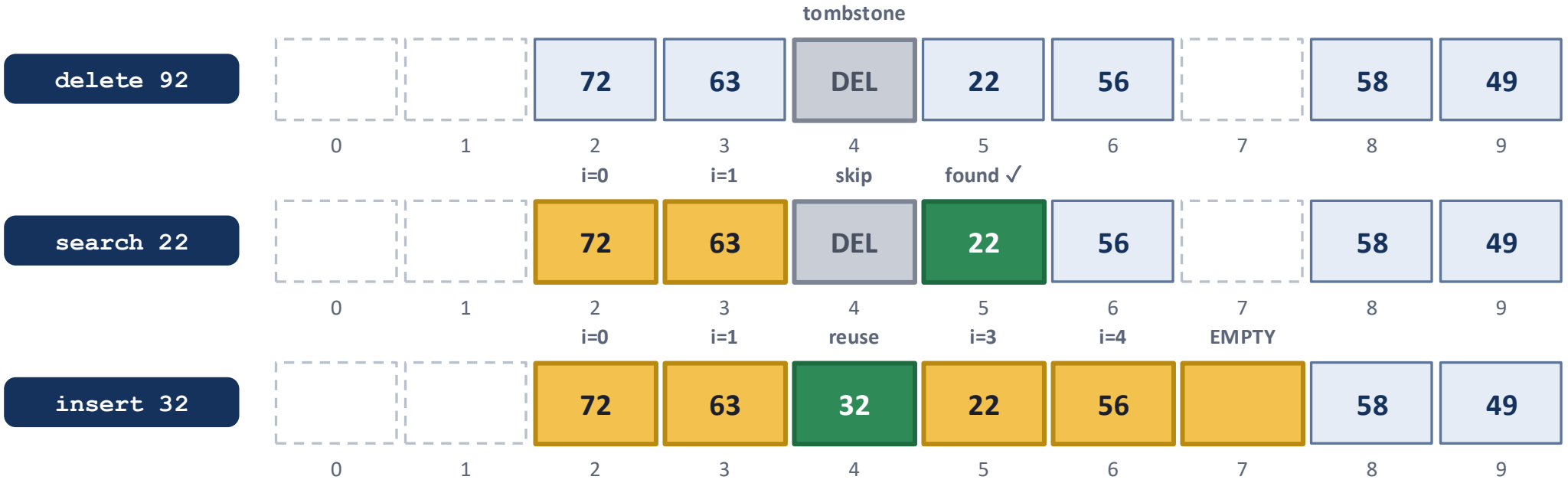
```
Original code:  table[index] = -1;   // Mark the slot as empty
```



False negative: 22 is still in slot 5, but the new EMPTY slot 4 cuts its probe chain. Running the original code: remove(72) makes search(92) return false — the same bug.

LO6 LO7 New

The fix: tombstones (DELETED ≠ EMPTY)



search EMPTY → stop (absent). DELETED → keep probing.

insert Remember the first DELETED slot, but keep probing until EMPTY (or m probes) to make sure the key is not already stored; then reuse that slot.

delete Find the key, set its state to DELETED.

LO6 LO7 Revised: s.31–34

Java (1/2): slot states, probe, bounded search

```
1  static final byte EMPTY = 0, OCCUPIED = 1, DELETED = 2;
2  int[] keys;   String[] values;
3  byte[] state;                    // all EMPTY initially
4
5  int probe(int key, int i) {      // linear probing
6      return (Math.floorMod(key, m) + i) % m;
7  }
8
9  int findSlot(int key) {          // index of key, or -1
10     for (int i = 0; i < m; i++) {          // <= m probes
11         int j = probe(key, i);
12         if (state[j] == EMPTY) return -1;     // absent
13         if (state[j] == OCCUPIED && keys[j] == key)
14             return j;                         // found
15         // DELETED or another key: keep probing
16     }
17     return -1;                               // tried all m
18 }
```

Line 3

state array: no key value is reserved, so −1 and Integer.MIN_VALUE are ordinary keys

Line 6

floorMod: negative keys get a valid home slot

Line 10

loop bounded by m: terminates even on a full table

Line 12

stop only at EMPTY; DELETED falls through and probing continues

LO6 LO7 Revised: s.32, 35

Java (2/2): insert with tombstone reuse, lazy delete

```java
void put(int key, String value) {
    int tomb = -1;                  // first DELETED seen
    for (int i = 0; i < m; i++) {
        int j = probe(key, i);
        if (state[j] == EMPTY) {    // key is absent
            store(tomb >= 0 ? tomb : j, key, value);
            return;
        }
        if (state[j] == DELETED) {
            if (tomb < 0) tomb = j; // remember, go on
        } else if (keys[j] == key) {
            values[j] = value;      // duplicate: update
            return;
        }
    }
    if (tomb >= 0) store(tomb, key, value);
    else throw new IllegalStateException("rehash");
}
```

```java
boolean remove(int key) {
    int j = findSlot(key);
    if (j < 0) return false;
    state[j] = DELETED;   // NOT EMPTY
    values[j] = null;
    size--;
    return true;
}
```

store(j, …)

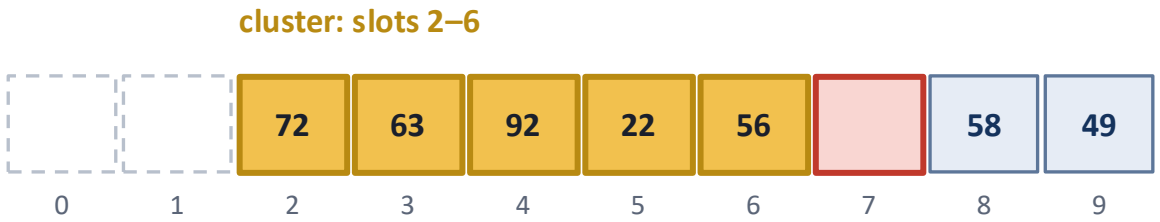
sets keys[j], values[j], state[j] = OCCUPIED and size++.

Full table or no free slot reached in m probes → exception: a real table rehashes into a bigger array long before this.

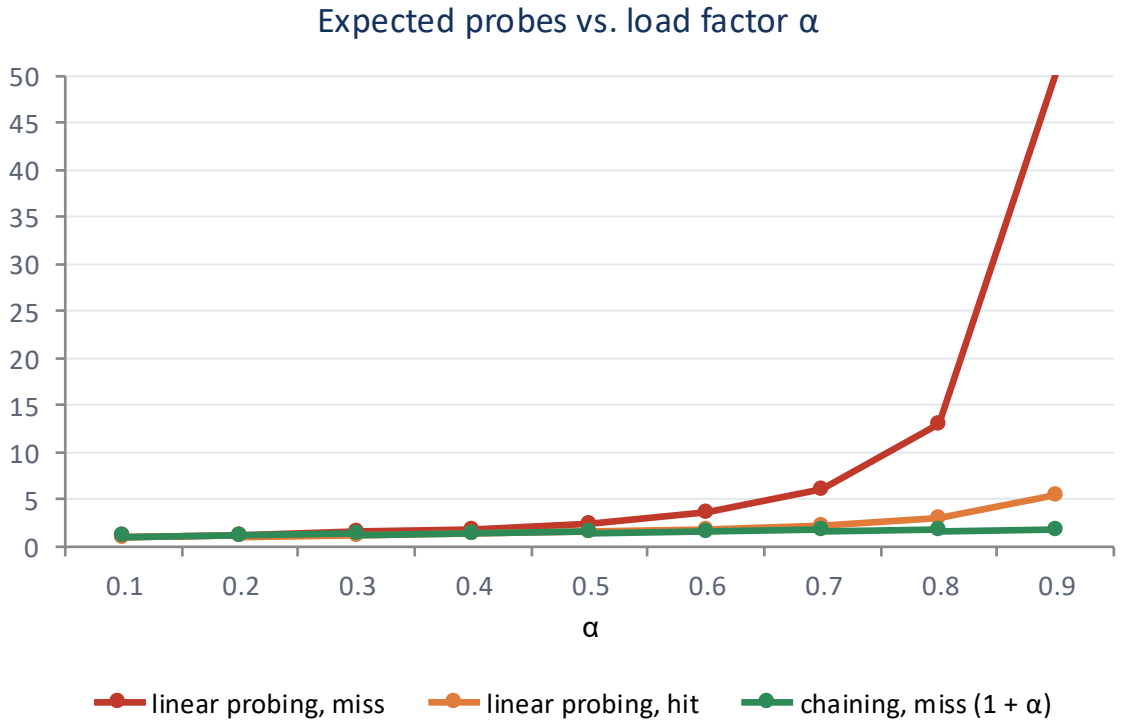
Duplicate policy: same key → update value. Full class: OpenAddressingHashTable.java + LinearProbingHashTable.java

LO6 LO8 New

Primary clustering: runs of occupied slots grow



Any new key with h(k) ∈ {2, …, 7} lands in slot 7: probability 6/10, versus 1/10 for an isolated slot. Clusters attract keys and merge.



Knuth's estimates (uniform hashing)

hit: $\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$

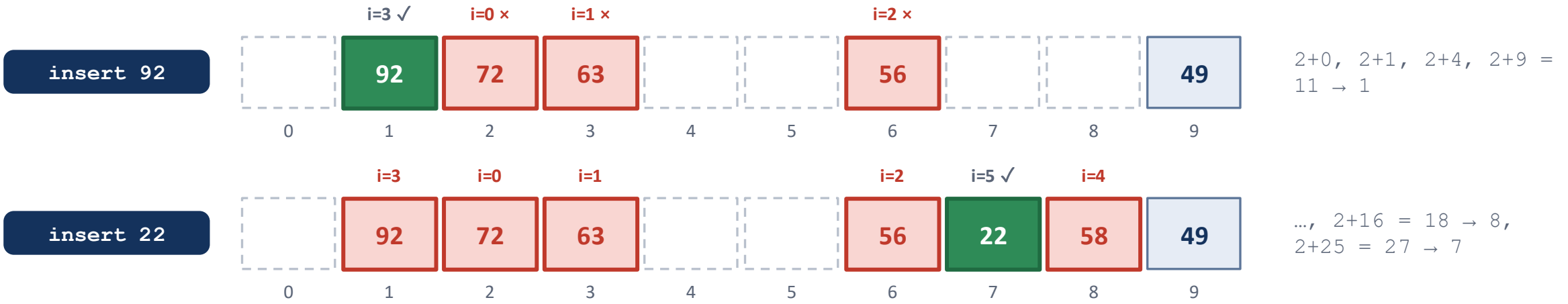
miss: $\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$

α = 0.5 → 1.5 / 2.5 probes; α = 0.9 → 5.5 / 50.5 probes. Keep α ≤ about 0.5–0.7 and rehash beyond that.

Quadratic probing: jump by i^2 from the home slot

$$m = 10,\ h(k, i) = (k \bmod 10 + i^2) \bmod 10,\ i = 0, 1, 2, \ldots \quad \text{offsets } 0, 1, 4, 9, 16, 25, \ldots$$

Same keys as linear probing: 49, 56, 72, 63 go to their home slots (9, 6, 2, 3).



Reduces primary clustering: 92 jumps out of the 2–3 block instead of extending it to slot 4.

Secondary clustering: keys with the same home (72, 92, 22) follow the same sequence 2, 3, 6, 1, 8, 7, … and collide with each other again.

LO6 LO7 Revised: s.37–40 (bug shown)

Quadratic probing may miss free slots

i^2 mod 10 only takes the values {0, 1, 4, 5, 6, 9}: from home 2 we can reach slots {2, 3, 6, 7, 8, 1} — never 0, 4, 5.



`probe sequence for 32: 2, 3, 6, 1, 8, 7, 8, 1, 6, 3, 2, 3, … (repeats forever)`

Original code: infinite loop

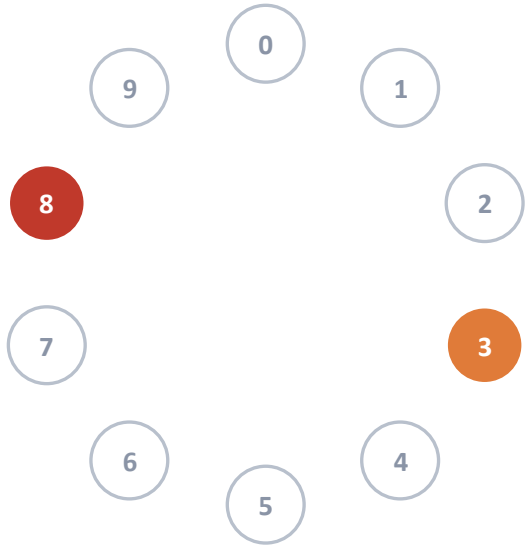
while (table[(index + i*i) % capacity] != -1) i++;
The guard size == capacity does not help: the table is only 70 % full.
We ran it — insert(32) never returns.

Correct approach

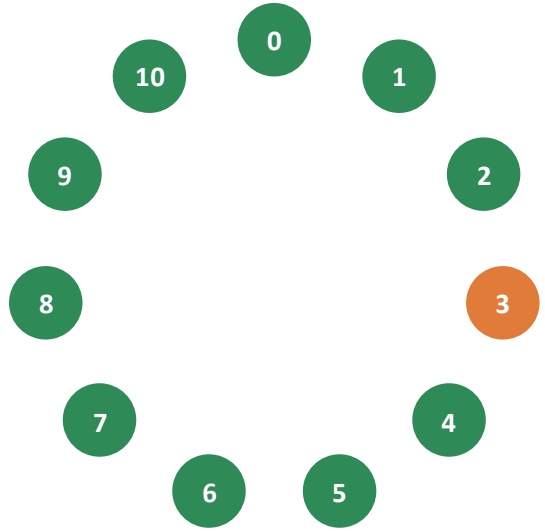
1. Bound the loop: at most m probes, then report "rehash".
2. Choose m prime and keep $\alpha \le \frac{1}{2}$: then the first $\lceil m/2 \rceil$ probes hit distinct slots, so an empty slot is always found.
3. Use long for i·i (overflow when i > 46,340).

Double hashing: the step size comes from a second hash

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \qquad h_1 = \text{home slot},\ h_2 = \text{step size}$$



m = 10, step 5: visits only {3, 8} ✗



m = 11, step 3: visits all 11 slots ✓

Requirements

$h_2(k) \neq 0$ otherwise every probe is the home slot.

$\gcd(h_2(k), m) = 1$ otherwise only m / gcd slots are ever visited.

Recipe: m prime, $h_2(k) = R - (k \bmod R)$ with $0 < R < m \Rightarrow 1 \le h_2 \le R < m \Rightarrow \gcd = 1$.

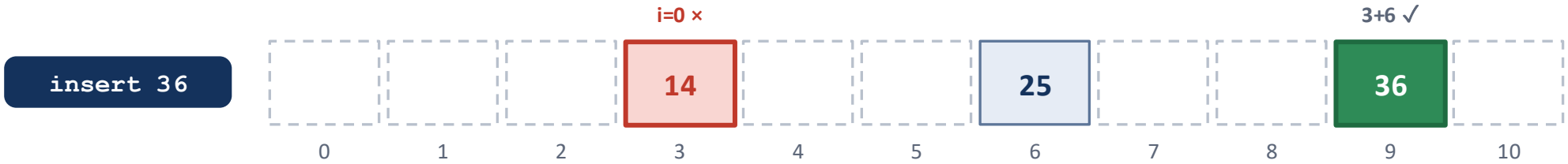
LO6 New

Double hashing trace: same home, different steps

$m = 11$ (prime), $h_1(k) = k \bmod 11$, $h_2(k) = 7 - (k \bmod 7)$

key	$h_1(k)$	$h_2(k)$ = step	probes	slot
14	3	7 − 0 = 7	3	**3**
25	3	7 − 4 = 3	3 ×, 6	**6**
36	3	7 − 1 = 6	3 ×, 9	**9**

All three keys have home slot 3, yet each needs at most 2 probes: their sequences leave slot 3 in different directions.



Full sequence for 25 (step 3): 3, 6, 9, 1, 4, 7, 10, 2, 5, 8, 0 — all 11 slots, each once.

EMPTY OCCUPIED DELETED probe collision inserted / found

Java: only probe() changes

```java
// Quadratic: offset i*i added to the HOME slot
int probe(int key, int i) {
    long offset = (long) i * i;  // no overflow
    return (int) ((home(key) + offset) % m);
}
```

Do NOT write this

```java
int index = home(key);
for (int i = 0; i < m; i++) {
    index = (index + i * i) % m;  // cumulative!
    ...
}
```

Offsets become 0, 1, 5, 14, 30, … — not the i^2 on the slide. Compute every probe from the home slot.

```java
// Double hashing: m prime, 0 < R < m
int h2(int key) {
    return R - Math.floorMod(key, R);   // 1..R
}

int probe(int key, int i) {
    long pos = home(key) + (long) i * h2(key);
    return (int) (pos % m);
}
```

Everything else is shared

findSlot, put, remove, the EMPTY / OCCUPIED / DELETED states, tombstone reuse and the m-probe bound are exactly the linear-probing code (OpenAddressingHashTable.java). The constructor of DoubleHashingHashTable rejects non-prime m and $R \notin (0, m)$.